

课程笔记

Building the best agentic analytics harness: Powered by Claude, built with Claude Code

Claude / Chris Merrick & Codex

2026 年 5 月 25 日



视频作者 / 频道: Claude

发布日期: 2026-05-21

视频时长: 26:46

视频链接: <https://www.youtube.com/watch?v=K4-flzsPraE>

字幕来源: YouTube 未提供字幕, 本笔记使用本地 Whisper 英文转写作为逐段复核来源。

目录

1 Claude Code 如何改变 Omni 的工程速度	4
1.1 Omni 与演讲主线	4
1.2 提交曲线背后的工程组织变化	4
1.3 从试用到稳定收益：Opus 与 Claude Code 的拐点	5
1.4 ShipIt、透明文化与公开 Demo	6
1.5 本章小结	6
2 从自然语言到语义查询的分析链路	7
2.1 用户问题进入系统	7
2.2 Claude 将问题翻译为语义查询	7
2.3 语义层到 SQL 的转换路径	8
2.4 业务语境为什么必须进入数据层	8
2.5 “Last Quarter” 示例：同一句话的多种业务含义	9
2.6 本章小结	9
3 语义层：数据策展、上下文定位与权限边界	10
3.1 语义层作为数据仓库之上的翻译层	10
3.2 数据策展：在海量表中指出可信路径	11
3.3 上下文定位：把说明放在字段定义旁边	12
3.4 权限边界与持续反馈	12
3.5 Blobby 的成长背景	13
3.6 本章小结	13
4 Blobby 演示：从问题分解到可视化回答	13
4.1 演示目标：向数据提问	13
4.2 PR 语义解析与 Semantic Model 检索	14
4.3 过滤值查找与 Fuzzy Matching	14
4.4 查询、运行、可视化与解释	15
4.5 本章小结	16
5 第一阶段：用 AI 上下文和样例查询补足元数据	16
5.1 单问单答版本的限制	16
5.2 Label 与 Description 的基础作用	17
5.3 AI Context：给 LLM 的使用说明	17

5.4	Sample Queries: 把典型问题锚定到查询模式	18
5.5	Values: 让模型看到字段值的形状	18
5.6	本章小结	19
6	第二阶段: Agentic Loop、错误恢复与 Sonnet 解锁	19
6.1	从问答到任务循环	19
6.2	自研 Agentic Harness 的工程投入	20
6.3	错误恢复预算与高质量错误消息	21
6.4	Evals 中的质量跃迁	21
6.5	从 Haiku 到 Sonnet: 复杂对话的模型选择	21
6.6	本章小结	22
7	第三阶段: Trace 诊断与“合并大脑”	22
7.1	CEO 压力如何推动质量工程	22
7.2	Trace: 观察 Agent 内部对话	23
7.3	Blobotomies: 从失败会话定位结构病灶	24
7.4	Outer Agent 与 Subagent 的 Split Brain	24
7.5	Consolidating the Brain: 把工具上提到外层 Harness	25
7.6	本章小结	25
8	第四阶段: 让 Claude 写 SQL, 再由 Omni 解析	25
8.1	为什么重新启用 SQL Parser	25
8.2	从 JSON 化查询切换到 SQL Parsing Mode	26
8.3	Parser Guidelines: 约束自由 SQL 的边界	27
8.4	One-shot Query 与 CTE 带来的效率	27
8.5	押注 Claude 的 SQL 能力	28
8.6	本章小结	28
9	当前 Harness: Checkpoint、工具面与 Evals	28
9.1	Outer Loop: Checkpoint 与故障恢复	28
9.2	Inner Loop: 不断扩展的工具集合	29
9.3	Dashboard、Visualization、Validation 与 Data Modeling	30
9.4	Evals 的双重价值: 质量度量与可观测性	31
9.5	Claude Code 如何反向塑造产品 Harness	31
9.6	本章小结	32

10 现场演示：AI 生成、UI 校验与排障	33
10.1 创建工程活动 Dashboard	33
10.2 Topic Discovery 与 Dashboard Plan	33
10.3 AI 生成，UI 校验与排障	34
10.4 Workbook 中检查 PR 数据与过滤条件	35
10.5 切换 Repository 与继续分析	36
10.6 预览 Dashboard：指标、作者与 PR 趋势	36
10.7 空白图表与现场排障	37
10.8 本章小结	38
11 总结与延伸：面向 Claude 优化的分析系统	38
11.1 演讲者的收束：Omni AI Analytics Platform Powered by Claude	38
11.2 全链路压缩：语义层、Harness、工具与 Evals	39
11.3 从工程经验到产品原则	40
11.4 实践清单：团队可以复用的设计问题	40
11.5 开放问题与下一步	41

1 Claude Code 如何改变 Omni 的工程速度

1.1 Omni 与演讲主线

Chris Merrick 在开场先把 Omni 的身份摆清楚：Omni 是一家 AI analytics platform 公司，目标是让业务用户可以直接“和数据说话”。视频说明里还给出一个很硬的背景事实：Chris 是 Omni 的 cofounder & CTO，Omni 平台约 99% 的代码由 Claude Code 辅助写成。这句话表面上像产品口号，真正的技术含义更硬：系统必须把人的自然语言、企业内部的业务语义、数据仓库里的表结构，以及最终可执行的 SQL 串成一条稳定链路。

本场演讲围绕两条线展开。第一条线是 Omni 如何用 Claude 和 Claude Code 改变自己的研发方式；第二条线是 Omni 如何用 Claude 构建面向分析任务的 agentic analytics harness。前者是“生产工具”的变化，后者是“产品能力”的变化。两条线在 Omni 身上合流：团队一边用 Claude Code 加速建造，一边把 Claude 放进产品内部，让它承担问题理解、查询生成、结果解释等工作。

什么是 analytics harness

这里的 *harness* 可以理解为给模型配套的执行外壳：它像一条有工位、有质检、有返工机制的装配线。Claude 负责理解和推理，harness 负责把模型接到工具、数据、状态、权限、错误恢复和评测体系上。没有 harness，模型只是一个会说话的“大脑”；有了 harness，大脑才有手、有眼、有记录本，也有犯错后继续修正的路径。

1.2 提交曲线背后的工程组织变化

Chris 作为 Omni 的 CTO 管理约 25 人工程团队。他展示的第一张关键幻灯片，是 Omni 主仓库 main branch 随时间变化的 commit 曲线。演讲者说这张图几乎“不言自明”：曲线的斜率明显变陡，意味着单位时间内进入主分支的变更更多。

这里容易误读。commit 数量无法直接等同于软件质量，更多提交也不自动意味着更好的产品。不过在一个成熟团队里，main branch 的提交曲线能反映组织吞吐量：需求被拆小、实现周期缩短、review 和合并节奏变快、工程师能更频繁地把可验证增量送进主线。换句话说，这条曲线像工厂里的传送带速度表，不能单独证明每件产品都合格，却能说明产线的运行节奏发生了变化。

更有意思的是，Chris 特别提到曲线里藏着他自己的 commit。作为一家面向数百客户的成长型公司 CTO，他原本以为自己迟早会停止写代码，把精力完全转到组织、客户和产品决策上。Claude Code 给他的意外收益是：他仍然能偶尔做软件工程。这段经历少一点“CTO 重新变回一线程序员”的浪漫色彩，多一点组织信号的现实意义：当工具降低了上下文切换和实现成本，高级技术人员可以把判断力更快地转化为代码变更。

提交速度的正确读法

commit velocity 应当被看作“组织把想法变成可合并代码的吞吐量”，不能被粗暴看成“工程质量”。它的教学价值在于揭示 Claude Code 改变了工程组织的摩擦：从理解任务、起草实现、修补边角，到让高级工程师重新参与局部编码，链路里的多个阻力都被降低了。

2-3x eng productivity with Claude Code

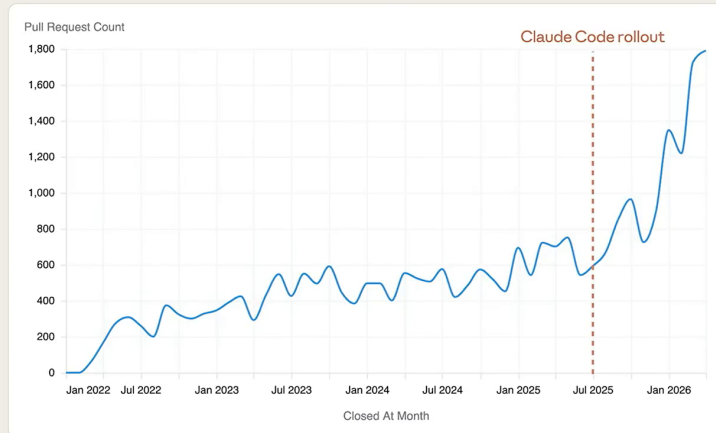


图 1: Claude Code 推出后，主分支 PR 数量曲线明显变陡，支撑演讲开头关于工程速度提升的核心论点。¹

1.3 从试用到稳定收益：Opus 与 Claude Code 的拐点

Omni 的采用路径并非一开始就线性成功。Chris 回忆，2025 年初团队内部的说法大致是：还不知道工作会在何时、以何种方式变化，但可以确定工作正在变化。因此团队先开始试用这些工具，主动寻找有效场景。

这个阶段有明显的 fits and starts，也就是时好时坏、断断续续。对工程团队而言，这很正常。新工具刚进入工作流时，真正的问题通常要落到真实代码库、真实约束和真实 review 标准中检验；demo 里写出代码只是门槛，长期稳定帮忙才算收益。小样本的惊艳很便宜，持续收益才昂贵。

拐点出现在 Claude Code 搭配 Opus 模型发布之后。Omni 的一些高级工程师开始说：这是真东西，它在持续帮忙。这个判断很关键，因为 senior engineer 的评估标准通常更苛刻。他们看重长期工程表现：上下文理解、修改可靠性、调试反馈、边界意识，以及能否接入日常工作流。

“能用”和“稳定有收益”之间隔着一层工程现实

早期 AI 编码工具常给人一种局部超能力：写样例、补模板、改小函数都很快。但真实收益要通过长期工作流检验，包括大型代码库上下文、隐含业务规则、测试失败后的修复能力、review 可读性，以及团队成员是否愿意反复使用。Omni 的叙述重点正是从试用热情走向稳定收益。

Chris 还描述了一个团队节奏上的变化：大家像是假期后在一月回来时突然完成了技能升级，开始真正“跑起来”。这句话背后有一个经常被忽略的变量：工具采用不只取决于模型能力，也取决于团队学习曲线。工程师需要学会拆任务、写提示、读模型输出、设置检查点、用测试约束生成结果，以及判断何时该接管。

¹ 视频画面时间区间：00:00:48-00:01:03。若为裁剪图，时间区间仍对应原视频画面。

1.4 ShipIt、透明文化与公开 Demo

速度在 Omni 并非孤立指标。Chris 把它连接到公司文化里的两个核心价值：ShipIt 和 transparency。ShipIt 强调持续交付，透明文化强调把建造过程公开给客户、潜在客户和社区看。

Omni 每周五有一次全员会议。会议前约 10 分钟用于公告和互相表扬，后面约 50 分钟，甚至越来越多时间，用来做 demo。所有 demo 会被录下来。Chris 还带着一点幽默说，CEO 最喜欢的工作是周六早上起床，把这些 demo 剪出来，发到 YouTube，再分享给外界。

这套机制让 Claude Code 带来的速度有了一个组织容器。没有固定展示节奏，速度很容易变成局部忙碌；有了每周 demo，速度被迫变成可观看、可解释、可传播的产品增量。透明文化也会反向约束质量：公开 demo 面向客户和社区的理解成本，规格高于普通内部状态汇报。

Demo 文化为什么能放大 AI 编码收益

AI 编码工具降低实现摩擦，但组织仍然需要决定“把新增速度用在哪里”。每周 demo 像节拍器：它让团队把增量压缩成可展示的故事，把工程产出转译成客户能看懂的变化。Claude Code 提供更快的手，ShipIt 和 demo 文化提供更清晰的方向盘。

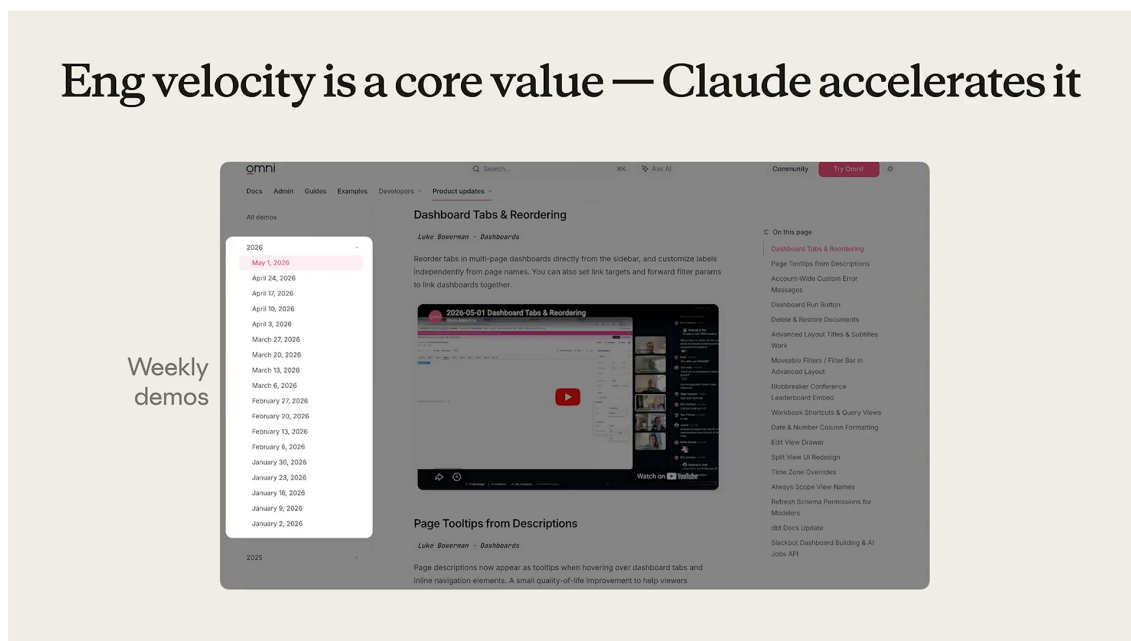


图 2: Omni 的公开周度 Demo 存档展示了 ShipIt 与透明文化如何被制度化为持续交付节奏。²

1.5 本章小结

本章的核心线索是：Claude Code 对 Omni 的影响首先表现为工程速度和组织节奏的变化。25 人工程团队的 main branch commit 曲线变陡，只是表层证据；更深层的变化是高级工程师确认工具进入稳定收益区间，CTO 也能在复杂职责之外重新参与局部编码。

这种速度没有悬空存在。Omni 用 ShipIt 价值观、每周全员 demo、公开视频发布，把更快的工程产出嵌入一套透明的交付文化里。因此，Claude Code 在这里已经超出单点工具范畴，成为被组织

² 视频画面时间区间：00:02:17–00:03:12。若为裁剪图，时间区间仍对应原视频画面。

节奏吸收后的生产力放大器。

2 从自然语言到语义查询的分析链路

2.1 用户问题进入系统

进入第二部分后，Chris 开始解释 Omni 用 Claude 建出的东西。Omni 的 AI analytics 允许用户直接向数据提问。用户输入的第一形态通常是一句自然语言问题，比如“上个季度合并了多少 PR”或“销售漏斗最近有什么变化”；SQL、BI 工具里的维度、指标和过滤器组合都属于后续系统翻译出来的结构。

自然语言的优势是接近人的意图，缺点是高度压缩。一个短句可能同时省略了时间口径、业务对象、数据来源、权限边界、聚合方式和展示形式。传统 BI 系统通常要求用户提前知道字段和过滤器；Omni 想做的事情，是让系统承担更多翻译工作，把人的问题逐步变成机器可以执行的查询。

自然语言查询的本质

自然语言查询不能理解成把一句话“直接丢给数据库”。更贴切的类比是：把一句业务请求交给一位懂公司术语、懂数据地图、懂 SQL 的分析师。Claude 扮演理解和翻译角色，语义层提供公司内部的数据地图，仓库执行最终计算。

2.2 Claude 将问题翻译为语义查询

在 Omni 的高层架构里，用户先提出问题，Claude 将问题翻译成 semantic query。这里的 semantic query 可以理解为“带业务意图的中间查询表示”。它尚未到最终 SQL 的精确度，却已经比原始自然语言更结构化：系统需要知道用户要看什么指标、按什么维度切分、使用哪段时间、需要什么过滤条件，以及这些概念对应语义层里的哪些对象。

为什么要有这个中间层？因为自然语言太松，SQL 又太硬。自然语言适合表达意图，SQL 适合表达精确执行计划。semantic query 位于两者之间，像报关单一样，把“我要寄一件东西”转成可检查、可路由、可生成下游指令的结构化信息。

这条链路可以形式化为：

$$q_{\text{user}} \xrightarrow{M} q_{\text{sem}} \xrightarrow{L_{\text{sem}}} q_{\text{sql}} \xrightarrow{W} r \xrightarrow{V} a$$

- q_{user} ：用户输入的自然语言问题。
- M ：Claude 模型，负责理解问题并生成中间语义意图。
- q_{sem} ：semantic query，即语义查询，承载指标、维度、时间、过滤器等业务化结构。
- L_{sem} ：semantic layer，即语义层，保存业务定义、字段含义、数据关系和使用规则。
- q_{sql} ：最终可在数据库或数据仓库中执行的 SQL 查询。
- W ：warehouse，即数据仓库或数据库执行环境，也可能是多个底层数据源。
- r ：查询返回的原始结果集。

- V：可视化与解释步骤，把结果整理成图表、摘要或后续分析对象。
- a：用户最终看到的分析回答。

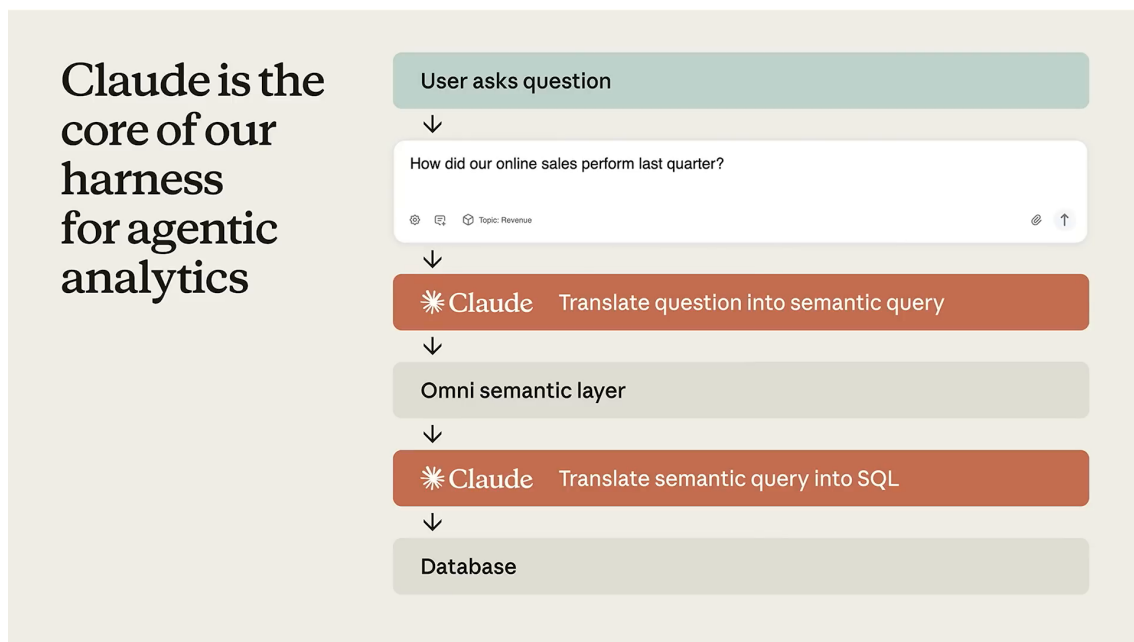


图 3: Omni AI Analytics 的高层链路：用户问题先由 Claude 转成语义查询，再经语义层约束并翻译为 SQL，最后在数据库中执行。³

2.3 语义层到 SQL 的转换路径

Chris 把 semantic layer 描述成位于数据仓库、数据库，甚至多个数据源之上的 translation layer。它的任务远超保存表名列表：它要给 Claude 一张能用的业务地图，说明哪些字段代表收入，哪些表应该被信任，不同对象之间怎样连接，哪些过滤条件合法，某个指标的定义是否随部门而变化。

从 semantic query 到 SQL 的转换，关键在于把业务词汇落到可执行结构上。举例来说，“customer”在一个系统里可能对应账户表，在另一个系统里可能对应合同实体；“active user”可能需要排除内部测试账号；“revenue”可能有 gross、net、recognized、booked 等多种口径。语义层就是把这些含义预先编码进系统，使模型不必凭常识猜测企业内部定义。

语义层像数据世界的城市地图

数据仓库像一座巨大城市，表、列、视图和历史遗留结构像街道、地下通道和旧楼。没有地图时，Claude 只能靠路名猜方向；有语义层时，系统能告诉它哪条路是主路、哪里禁止通行、哪些地点名称在公司内部有特殊含义。地图越贴近业务现实，生成 SQL 的路径越少走弯路。

2.4 业务语境为什么必须进入数据层

Chris 明确指出，Claude 非常擅长回答问题，也能讲出许多关于一般商业运作的深刻见解。但如果希望它回答“这家公司”的问题，就必须告诉它这家公司怎样运转：使用哪些术语、业务对象如何

³视频画面时间区间：00:03:19–00:04:06。若为裁剪图，时间区间仍对应原视频画面。

定义、数据长什么样、字段怎样连接、哪些口径有组织差异。

这里的难点很细。LLM 的通用知识像一本百科全书，能解释什么是销售、季度、客户、收入；企业数据问题需要的是一本内部操作手册，写着“本公司销售团队说 quarter 时指财年季度，产品工程团队说 quarter 时指自然年季度”。如果这类语境只留在人脑、Slack 讨论或零散文档里，模型生成的答案就会看似流畅，实际偏离业务。

通用商业知识会制造“看起来合理”的错误

LLM 可以根据常识补全缺失信息，这正是危险之处。数据分析里的错误常常不会表现为语法错误，更多会表现为口径错、时间窗错、过滤条件错、使用了过时表。答案读起来很顺，数字却在业务上站不住。语义层的价值就是把这些隐性约定搬进可执行的数据定义里。

2.5 “Last Quarter” 示例：同一句话的多种业务含义

Chris 用“last quarter”说明问题的微妙性。同一句话在不同公司可能含义完全不同；甚至在 Omni 内部，不同组织也有不同解释。产品和工程团队说 last quarter，通常指自然年季度；销售团队说 last quarter，则指公司的财年季度。

这个例子小，却足够锋利。假设现在是 5 月，产品工程团队问 last quarter，可能指 1 月到 3 月；销售团队问 last quarter，如果公司财年从 2 月开始，可能指 2 月到 4 月。两个答案都能自治，SQL 都可能执行成功，图表都可能画得漂亮。唯一的问题是：如果系统不知道提问者所在业务语境，它很难知道哪一个才是正确答案。

因此，时间词、部门语境和数据定义必须被编码到 context、awareness，甚至 data layer 和数据定义中。对 AI 分析系统来说，业务语义属于查询正确性的一部分，不能降格成旁注。

“Last Quarter” 的教训

自然语言里的一个短词，可能映射到多套业务规则。分析系统必须把这些规则显式放进语义层：谁在问、问的是哪个业务域、该域使用哪套时间口径、最终 SQL 应该如何落到具体日期范围。缺少这层编码，模型越会补全，越可能自信地产生错误口径。

2.6 本章小结

本章建立了 Omni AI analytics 的第一条技术链路：用户用自然语言提问，Claude 将问题翻译成 semantic query，语义层把业务意图映射到数据仓库可执行的 SQL，仓库返回结果，再由系统组织成用户能理解的分析回答。

这条链路的难点不在 SQL 语法本身，而在业务语境进入数据层的方式。术语、时间口径、字段定义和部门差异如果没有被结构化保存，模型只能依赖通用知识和猜测。Omni 的方向是把这些隐性业务规则沉淀到语义层，让 Claude 的语言能力和企业数据的真实结构真正接上。

3 语义层：数据策展、上下文定位与权限边界

3.1 语义层作为数据仓库之上的翻译层

上一章已经把自然语言问题、Claude、语义查询、SQL 与数据仓库串成了一条链。本章进一步放大其中最容易被低估的一环：**语义层**。Chris Merrick 在 00:05:07 处给出简洁定义：语义层是位于数据库之上的翻译层。这里的“翻译”并非简单把英文句子改写成 SQL 语句，它更像企业数据的业务词典、导航地图和交通规则的结合体。

数据仓库保存的是表、字段、类型、分区、主键、外键和历史数据；业务人员提问时使用的是收入、机会、客户、仓库、PR、季度、合并状态等概念。两者之间天然隔着一层语义间隙。语义层的作用，就是把“人真正想问什么”落到“系统应该查哪张表、用哪个字段、怎样连接、如何过滤、谁有权限看”的可执行结构上。

语义层的直觉：数据库上方的业务机场塔台

可以把数据仓库想成一座巨大机场：跑道很多，航站楼很多，飞机和车辆也很多。SQL 像具体的驾驶指令，能让某架飞机从某条跑道起飞；语义层像塔台，知道哪些跑道开放、哪条路径可信、哪些区域需要许可、哪些航班名称和真实编号对应。缺少塔台时，LLM 也许能写出语法正确的 SQL，却难以稳定判断哪条业务路径值得信任。

若把这件事形式化，可以把用户问题 q 经过模型理解后映射到一个语义查询 s ，再由语义层编译为 SQL x ：

$$q \xrightarrow{\text{Claude}} s \xrightarrow{\text{Semantic Layer}} x \xrightarrow{\text{Warehouse}} y$$

- q ：用户用自然语言提出的问题，例如“某个仓库最近合并的 PR 情况如何”。
- s ：语义查询，表达业务意图、指标、维度、过滤条件和时间范围。
- x ：可运行的 SQL 或等价查询计划。
- y ：数据仓库返回的结果集，随后可被解释、可视化或继续分析。

这条链里，语义层承担的是把业务世界压缩成可计算世界的工作。压缩的质量决定了后续 agent 是否能稳定落地。

Omni's semantic layer steers Claude

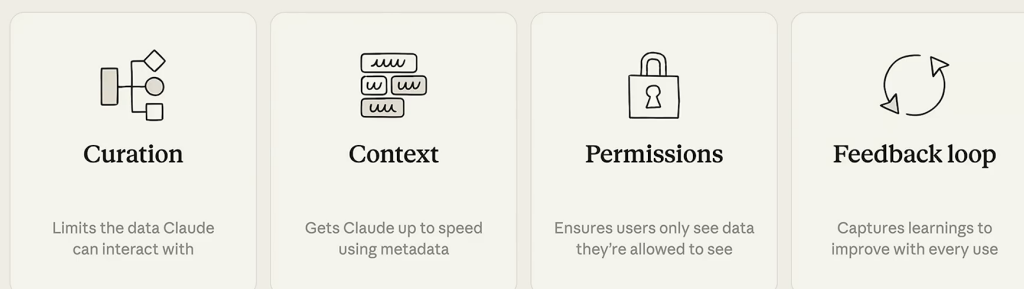


图 4: 语义层用数据策展、局部上下文、权限和反馈循环四个控制面,把企业数据的业务规则交给 Claude 使用。⁴

3.2 数据策展：在海量表中指出可信路径

演讲者强调，玩具演示与真实企业数据之间差距巨大。一个十来张表的小数据库里，LLM 或经验丰富的人类分析师很容易找到正确答案；真实公司的数据仓库经常有数万、数十万甚至更多数据集。更麻烦的是，表名和字段名会产生大量近义、历史遗留和局部团队命名：可能存在一百张 revenue 表，也可能存在一百张 opportunity 表。每一张都看起来合理，只有少数几张真的服务于当前业务口径。

数据策展指的就是在这片表海里标出可信路径。策展这个词很准确：博物馆并不会把仓库里所有藏品摊在大厅里，它会选择展品、组织动线、写清标签，并告诉观众哪些关系值得看。语义层对数据做的事情类似：它告诉 Blobby 哪些表应该被优先使用，哪些表是旧口径、临时表、实验表或边缘团队产物。

语义层的第一项核心职责

语义层把“所有可查的数据”收束为“当前问题应该使用的数据”。对于 agent 而言，这一步相当于减少搜索空间：没有策展时，模型面对的是海量可能路径；有策展时，模型先获得一张经过业务确认的路线图。

这里还有一个隐藏问题：表与表之间如何拼接。真实分析很少只查一张表，常见问题会跨客户、销售机会、仓库、提交记录、时间窗口和组织结构。语义层保存的不只是字段解释，也包括哪些数据可以一起使用、哪些连接关系可信、哪些过滤条件属于标准口径。它让 agent 少走“语法正确、业务错误”的弯路。

⁴视频画面时间区间：00:05:07-00:07:17。若为裁剪图，时间区间仍对应原视频画面。

3.3 上下文定位：把说明放在字段定义旁边

语义层的第二项能力是编码上下文。Chris Merrick 先肯定上下文本身重要，随后给出更关键的限定：上下文越靠近它所解释的定义，效果越好。他用 Claude Code 的 `Claude.md` 做类比：如果项目说明散落在远处的大文件里，模型仍然需要猜哪些说明适用于哪段代码；如果说明贴近对应模块、目录或关键代码，模型就更容易正确使用它。

在数据世界里，同样的原则适用于字段定义。例如，一个字段叫 `status`，它可以表示订单状态、PR 状态、销售机会阶段，也可能包含历史枚举值。把“这个字段在什么场景下可用、哪些值意味着完成、哪些值已经废弃、哪些团队沿用旧称呼”放在字段定义旁边，模型拿到字段时就能同时拿到使用说明。

上下文和定义分离会制造隐性歧义

把大量说明放进一个远离字段定义的总说明文件，会让上下文变成“背景噪声”。模型可能读到正确说明，却无法稳定判断它适用于哪个字段；也可能把一个业务域的规则误套到另一个业务域。语义层的价值之一，就是让说明贴着字段、模型、关系和过滤值出现，降低错配概率。

这种“贴近定义”的设计对 LLM 尤其重要。传统程序可以用强类型、外键和测试来约束行为；LLM 的判断更依赖上下文提示。上下文离目标对象越远，模型越容易把相似词当成同一概念。上下文贴近字段后，模型面对的就不再只有孤零零的列名；它同时获得一组带解释、边界和使用规则的业务对象。

3.4 权限边界与持续反馈

语义层的第三项职责是权限。演讲者讲得很直接：让用户看到他们应该看到的数据，不看到他们不应该看到的数据。对于分析 agent 来说，权限不能停留在网页按钮上的附属功能层面，它必须嵌入查询路径本身。因为 agent 会自动生成查询、选择字段、运行 SQL、总结结果；如果权限只在最后展示阶段才检查，敏感数据已经可能进入中间结果、trace 或模型上下文。

可以把权限抽象为一个访问函数：

$$A(u, r, a) \in \{0, 1\}$$

- u ：发起请求的用户或用户所属角色。
- r ：被访问的数据资源，例如表、字段、行级数据或派生指标。
- a ：动作，例如查询、聚合、导出或写回语义定义。
- $A(u, r, a) = 1$ ：允许该动作； $A(u, r, a) = 0$ ：拒绝该动作。

这只是最小抽象。真实系统里，权限还会包含组织层级、客户隔离、行级过滤、字段脱敏和审计记录。语义层把这些规则和业务定义放在同一套数据地图中，Bobby 才能在“理解问题”的同时理解“这件事能不能做”。

演讲者还强调反馈循环。真实组织里的数据定义会不断变化：新字段出现，旧表退役，团队调整口径，业务指标被重新定义。Omni 的应用会把后续提问反馈回定义和上下文，让下一次问题受益于这次使用经验。这个闭环让语义层从静态文档变成可持续更新的系统资产。

反馈循环的工程意义

反馈循环让语义层跟上组织变化。用户的问题暴露了定义缺口，系统把缺口转化为字段说明、模型关系、过滤值样本或权限规则的改进。下一次 agent 遇到相似问题时，起点更接近正确业务口径。

3.5 Blobby 的成长背景

在铺垫完语义层后，演讲者引出 Omni 的 AI 数据分析 agent: **Blobby**。他用带一点玩笑的方式称它现在是一位成熟、专业、精致的数据分析师。这个说法的重要性不在幽默本身，而在时间线: Blobby 大约从 18 个月前开始构建，期间经历了多个阶段，能力和质量逐步提高。

本章只负责定义 Blobby 的角色: 它是 Omni 平台中的 AI 数据分析 agent，围绕 Claude、语义层、查询生成、数据仓库执行、可视化和解释组成分析 workflow。后续章节会展开它如何从早期问答形态演进到更完整的 agentic harness。本章需要记住的事实很简单: Blobby 的能力并非只来自“模型变聪明”，还来自语义层、权限、反馈循环和产品界面共同提供的环境。

3.6 本章小结

本章定义了语义层、语义模型、权限边界、反馈循环和 Blobby 的基本位置。语义层位于数据仓库之上，负责把业务语言翻译为可执行的数据路径; 它通过数据策展缩小表海搜索空间，通过贴近字段定义的上下文减少误解，通过权限规则保护数据边界，并通过持续反馈保持定义新鲜。Blobby 的成熟，正建立在这套业务语义基础之上。没有这层基础，agent 也许能写 SQL，却很难稳定回答真实公司的业务问题。

4 Blobby 演示: 从问题分解到可视化回答

4.1 演示目标: 向数据提问

从 00:07:55 开始，演讲者播放一段简短演示，用来把前面关于语义层和 Blobby 的讨论落到真实产品体验上。演示目标很朴素: 用户直接向自己的数据提问，Blobby 等待问题、拆解问题、寻找相关数据、生成查询、运行查询，并把结果解释给用户。

这个片段很重要，因为它第一次展示了 PR 到答案的完整操作链。用户的问题涉及 PR，也就是 GitHub 的 pull request，通常指代码变更提交后请求合并的协作对象。对于人类工程团队而言，“PR”是日常缩写; 对于数据系统而言，它需要被解析成某个语义模型中的对象、字段和过滤条件。



图 5: Blobby 作为 Omni 的分析 agent 被正式引入，后续演示和架构改造都围绕它展开。⁵

4.2 PR 语义解析与 Semantic Model 检索

Blobby 的第一步是理解问题中关键名词的业务含义。演讲者指出, Blobby 知道 PR 指的是 GitHub pull request, 因为 Claude 能理解这个缩写工程语境中的常见含义。理解缩写只是开头, 真正的动作是到语义模型中寻找对应的数据对象。

语义模型可以理解为语义层内部的一张结构化业务地图。它记录某个业务域有哪些对象、字段、关系、指标、过滤条件和可用值。Blobby 确认 “PR = GitHub pull request” 后, 需要继续定位: 这个概念在 Omni 的数据里对应哪个 topic、哪张表、哪些字段、哪些状态值, 以及怎样和仓库、时间、作者或合并状态连接。

这里可以看到语义层和 LLM 的分工。Claude 负责把自然语言中的 “PR” 识别成业务概念; 语义模型负责告诉系统这个概念在企业数据资产中的落点。只有两者结合, Blobby 才能从 “听懂用户在说什么” 走到 “知道该查哪份数据”。

4.3 过滤值查找与 Fuzzy Matching

演示中的问题还限定了一个特定 repository, 因此 Blobby 必须应用过滤条件。过滤条件看起来只是 SQL 里的 WHERE, 实际需要先找到合法过滤值。用户输入的仓库名可能大小写不同、包含缩写、漏字、多字或拼写错误。演讲者直接承认, 自己向 LLM 提问时也经常打错字, 所以系统需要 fuzzy matching。

模糊匹配指的是当用户输入 z 与合法值集合 V 中的值不完全一致时, 系统寻找最接近候选值的过程:

$$v^* = \arg \max_{v \in V} \text{sim}(z, v)$$

⁵ 视频画面时间区间: 00:07:18-00:07:53。若为裁剪图, 时间区间仍对应原视频画面。

- z : 用户输入的过滤值，例如某个仓库名的口语写法或拼写错误版本。
- V : 语义模型或数据集中可用的合法过滤值集合。
- $\text{sim}(z, v)$: 相似度函数，可以来自编辑距离、向量相似度、规则匹配或多种方法组合。
- v^* : 最可能对应用户意图的标准值。

这一步看似小，却决定了回答是否落到正确数据范围。过滤值错了，后续 SQL 再漂亮也只是在错误集合上做精致计算。

过滤值比字段名更容易被低估

很多分析系统只强调“找对表、找对字段”，却忽视字段值本身也需要语义定位。真实用户不会总是输入标准枚举值；他们会输入简称、旧称、错别字、大小写混合或团队内部黑话。没有过滤值查找和模糊匹配，agent 很容易在最后一公里跑偏。

4.4 查询、运行、可视化与解释

完成概念解析、语义模型检索和过滤值定位后，Blobby 才进入执行阶段。演示中的流程是：生成查询，把查询发送到数据仓库运行，拿回结果，生成可视化，最后用简短总结告诉用户图里展示的含义。

这一连串步骤可以压缩为一条端到端流水线：

Blobby 演示中的端到端流水线

用户问题 → 识别 PR 为 GitHub pull request → 在 semantic model 中定位数据对象 → 查找 repository 过滤值 → 对用户输入做 fuzzy matching → 生成查询 → 在数据仓库运行 → 返回结果 → 生成可视化 → 给出文字解释。

这条流水线说明了 Blobby 的产品价值：用户不需要先知道表名、字段名、枚举值和连接关系，也不需要手写 SQL。系统把自然语言问题拆成可执行的若干微动作，并在每一步借助语义层和工具完成校准。与此同时，最终答案不只是一段文字，还包含图表和解释，这让用户能够更快判断结果是否符合业务直觉。

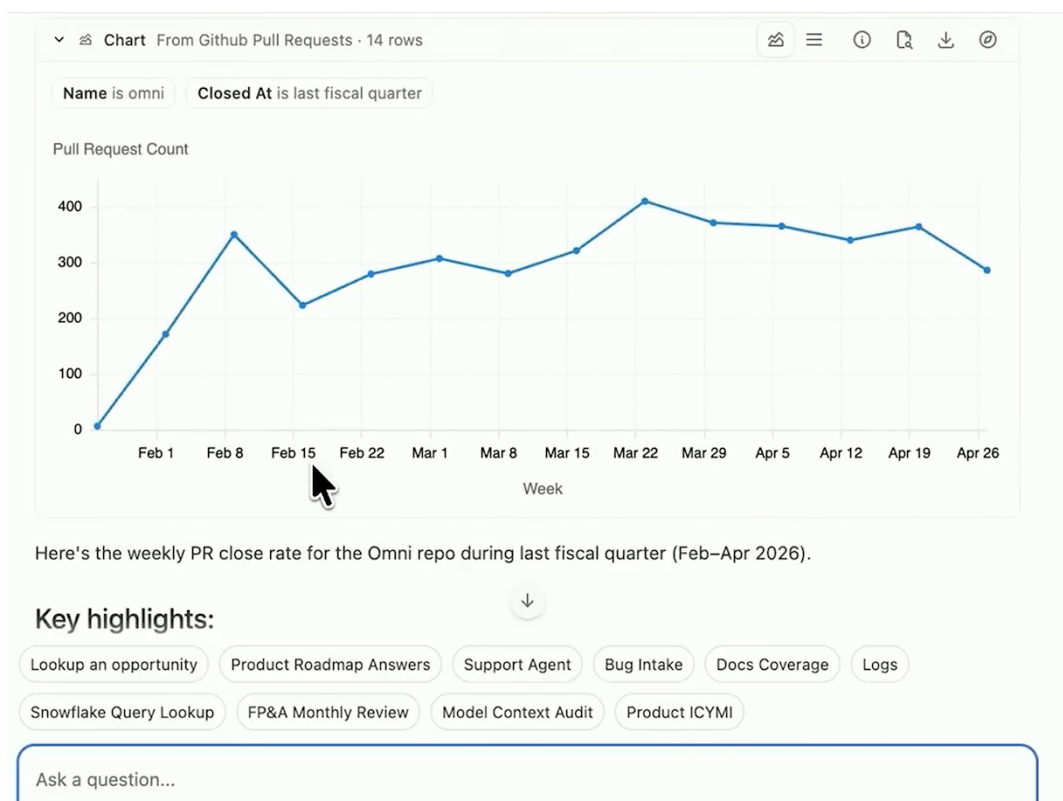


图 6: Blobby 的录制演示最终生成带过滤条件的 PR 趋势图和文字解释，展示从自然语言问题到可视化回答的完整路径。⁶

4.5 本章小结

本章用 Blobby 的第一次操作演示串起了前面定义的概念。Blobby 面对一个关于 PR 的问题，先用 Claude 的语言理解能力解析缩写，再用 semantic model 找到企业数据中的对应对象；随后它为特定 repository 查找过滤值，并用 fuzzy matching 修正用户输入中的不精确表达；最后生成查询、运行数据仓库、构建可视化并给出解释。这个演示展示的核心不在单次问答的炫技，它更在一条可复用的分析路径：语言理解、语义定位、权限与过滤约束、查询执行、可视化表达和结果解释必须连续工作，用户才会感觉自己真的在“向数据提问”。

5 第一阶段：用 AI 上下文和样例查询补足元数据

5.1 单问单答版本的限制

Blobby 的第一版很朴素：用户提出一个自然语言问题，系统给出一个答案。这个形态能证明方向可行，却很快撞到企业数据的真实墙面。企业数据表面上是字段、表、过滤条件，实际使用时还有团队习惯、命名惯例、业务边界和隐含口径。一个字段名像门牌号，只能告诉模型“这里有一扇门”；模型还需要知道门后是会议室、仓库，还是财务档案柜。

演讲者强调，他们很快发现需要给系统更多 metadata，即“关于数据的数据”。这里的元数据并非只服务人类阅读，也要服务 LLM 的推理和工具调用。数据团队与管理员需要把“这份数据通常怎

⁶视频画面时间区间：00:07:55-00:09:21。若为裁剪图，时间区间仍对应原视频画面。

么用”“遇到某类问题应该查哪个字段”“哪些值是合法缩写”这些经验写到语义定义旁边，让模型少猜、多对齐。

关键概念

单问单答版本的根本限制：它把分析任务压缩成一次输入和一次输出，却没有给模型足够的业务地图。对于 AI 分析 agent，问题质量由用户输入决定，答案质量还取决于语义层提供的可用上下文。

5.2 Label 与 Description 的基础作用

Omni 原本已经在 definitions 里维护 **label** 和 **description**。label 是字段或对象的短名称，像图书馆书脊上的标题；description 是给人看的说明，解释这个字段大致表示什么、什么时候该用。它们是语义层的基础入口，帮助模型把自然语言里的“region”“repository”“PR”映射到系统内部对象。

可是基础说明仍然偏静态。label 能告诉模型字段叫“Region”，description 能说明它是销售区域或运营区域；它们未必告诉模型，当用户说“United States”时，过滤值也许应写成“U.S.”，当用户问某类收入问题时，也许应优先走某个经过数据团队验证的指标。换句话说，label 和 description 解决“这是什么”，更细的 AI 使用说明解决“在具体问法里怎样用”。

背景知识

元数据的层次感：label 像名片，description 像一句简历，AI context 与 sample queries 像资深同事在旁边补充的实战提醒。LLM 读到这些提醒后，更容易把用户问题落到正确字段、正确过滤值和正确查询模式上。

5.3 AI Context：给 LLM 的使用说明

于是 Omni 增加了 **AI context** 概念。AI context 是专门写给 LLM 的字段级或对象级使用说明，用来回答“当你被问到这个数据时，应该怎样理解、怎样选择、怎样避免误用”。它可以提示模型使用某个 reference，优先参考某个 field，或者在某类问题里采用某个经过验证的路径。

这一步很关键，因为 LLM 的弱点常常出现在“看起来合理”的地方。它可能知道字段名，也能读懂描述，却会在多个近似字段之间选错一个。AI context 的作用像铁路道岔：火车本身有动力，但道岔决定它进入哪条轨道。对于数据管理员来说，这相当于把经验规则直接贴在语义定义上，让模型在运行时拿到局部、具体、可执行的指令。

关键概念

AI context 的定义：AI context 是绑定在语义层定义上的 LLM 使用说明。它既解释字段含义，也指导模型在用户问题、字段选择、参考对象和过滤条件之间做更可靠的映射。



图 7: 扩展元数据把 label、description、AI context、sample queries 和 values 放在同一张图上, 说明 LLM 需要哪些局部操作提示。⁷

5.4 Sample Queries: 把典型问题锚定到查询模式

Sample queries 指典型业务问题与查询写法的示例。演讲者说它很直观: 告诉模型“这是常见用例, 遇到像 X 这样的问题时, 可以运行这样的查询”。样例查询的价值在于给 LLM 一个可模仿的结构, 而模仿在这里很有用, 因为企业分析往往有稳定模式: 按时间聚合、按地区切分、过滤某个产品线、比较当前期与上一期。

可以把 sample queries 理解成“查询谱系里的样板间”。用户问法可能变化很多, 例如“本季度欧洲收入怎样”“EMEA 最近增长是否变慢”“按区域看 pipeline 变化”。如果这些问题都应该落到类似的时间过滤、地区维度和指标计算上, 样例查询就能给模型一个锚点。锚点越清晰, 模型越少依赖自由发挥。

易错点

样例查询的边界: sample queries 的目标是展示典型路径, 无须穷举所有问法。写得太少, 模型缺少参照; 写得太散, 模型会被互相冲突的模式拉扯。好的样例应覆盖高频、关键、容易误解的业务问题。

5.5 Values: 让模型看到字段值的形状

最后一个细节是 **values**, 也就是字段值样本。演讲者说这点更 subtle: 模型看到字段名“region”还不够, 它最好看到一些代表性值, 比如 EMEA、NAM、APAC。看到这些值后, 模型可以推断它面对的是世界区域缩写体系, 理解会超出普通文本标签。

字段值样本 V_f 可以写成:

⁷ 视频画面时间区间: 00:09:29-00:11:06。若为裁剪图, 时间区间仍对应原视频画面。

$$V_f = \{v_1, v_2, \dots, v_k\}$$

其中：

- f ：某个字段，例如 `region` 或 `repository`。
- V_f ：字段 f 的代表性取值集合。
- v_i ：一个具体取值，例如 `EMEA`、`NAM`、`APAC`。
- k ：展示给模型的样本数量，通常只需覆盖形状和常见枚举。

这组值让模型学到“值的形状”。如果用户说 `United States`，系统中的合法值可能是 `U.S.`；如果用户说 `North America`，系统里可能用 `NAM`。值样本把自然语言和数据库枚举之间的距离缩短了。它也能辅助模糊匹配：当用户拼写不精确、使用全称、缩写或口语表达时，模型更容易找到合法过滤值。

背景知识

Values 与 schema 的差别：schema 告诉模型字段有哪些列，values 告诉模型这些列里长什么样。前者像表格的表头，后者像几行真实样本。对过滤条件、枚举值、地区缩写、仓库名、产品线名称而言，后者常常决定查询能不能落地。

5.6 本章小结

第一阶段的教训是：AI 分析质量不能只靠模型“聪明”。Omni 给语义定义补上 `label`、`description` 之外的 `AI context`、`sample queries` 和 `values`，本质是在语义层中加入可执行的使用经验。`label` 帮模型定位对象，`description` 解释含义，`AI context` 指明用法，`sample queries` 提供查询范式，`values` 展示合法取值的形状。它们共同把自然语言问题拉近到可执行查询。

6 第二阶段：Agentic Loop、错误恢复与 Sonnet 解锁

6.1 从问答到任务循环

补足元数据以后，Bobby 的问答质量明显提升，但演讲者指出，此时它仍然不算真正的 `agent`。关键跃迁是给它加上 **agentic loop**，也就是让系统围绕一个任务持续规划、调用工具、观察结果、修正动作。单次问答像一次投篮；`agentic loop` 更像一轮进攻战术：控球、传导、观察防守、调整路线，直到拿到可用结果。

task 是这个循环里的工作单元。它把用户的笼统目标拆成可以推进的步骤，例如寻找相关语义模型、生成查询、运行查询、解释结果、修复失败。一个任务不一定一次成功，重要的是 `harness` 能记录当前状态，并允许 `agent` 根据观察继续行动。

可以把这一阶段的 `harness` 简化为：

$$H = (M, T, B_e, \tau)$$

其中：

- H : agentic harness，即围绕模型构建的执行框架。
- M : 被调用的 Claude 模型，例如 Haiku 或 Sonnet。
- $T = \{t_1, \dots, t_n\}$: 可用工具集合，例如查询、执行、校验、可视化工具。
- B_e : 错误恢复预算，表示允许 agent 为修复失败消耗的轮次、时间或 token 空间。
- τ : 运行轨迹，记录模型消息、工具调用、错误和观察结果。

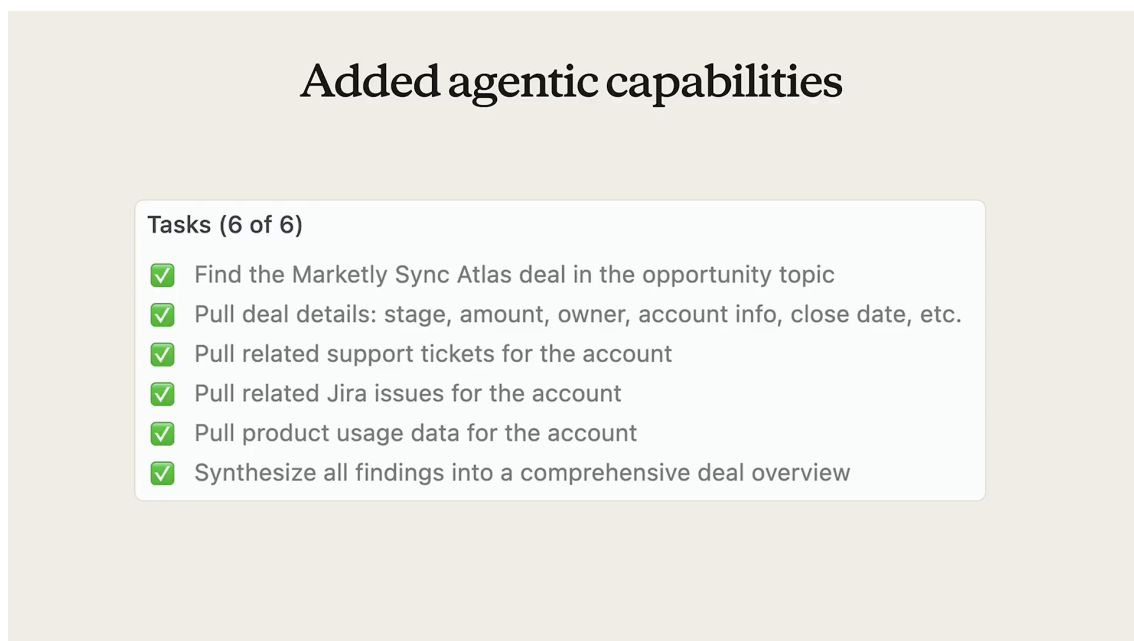


图 8: agentic loop 将分析请求拆成可恢复的任务步骤，使系统能查询数据、处理错误并逐步合成答案。

8

6.2 自研 Agentic Harness 的工程投入

Omni 为这一跃迁自研了 agentic harness。演讲者称这是一个 big engineering effort，里面包含任务概念，也包含其他支撑机制。harness 这个词很形象：它像安全带和工装夹具，把模型固定在可控的执行环境里。模型负责推理，harness 负责把推理接到工具、状态、错误处理和评测上。

这里有一个容易低估的事实：agent 能力很少在“给模型更多工具”之后自动出现。工具越多，状态越复杂，错误路径也越多。harness 的工程价值在于规定循环如何前进、失败如何反馈、上下文如何保留、何时重试、何时停止。没有这些机制，模型可能只是连续调用工具；有了这些机制，系统才开始具备任务推进能力。

关键概念

agentic loop 的定义：agentic loop 是 agent 在目标约束下反复执行“计划 → 工具调用 → 观察 → 修正”的过程。它把一次回答扩展为多步任务执行，也把错误恢复变成系统能力。

⁸视频画面时间区间：00:11:09-00:12:08。若为裁剪图，时间区间仍对应原视频画面。

6.3 错误恢复预算与高质量错误消息

这一阶段最早的大幅质量提升，来自错误恢复。Omni 做了两件事：第一，明确告诉 Blobby 如何从错误中恢复，并给它一定预算去做；第二，投入精力提供更好的错误消息，让错误消息不仅描述发生了什么，还提示可能怎样修复。

error recovery 是失败后的自我修复能力；**error budget** 是留给这种修复的空间。这个预算可以理解为登山时预留的备用绳和补给。没有预算，agent 第一次滑倒就结束；预算过大，agent 可能在无望路径上反复消耗资源。合理预算让系统有机会改正可修复错误，又能避免无限重试。

错误消息质量同样关键。低质量错误消息只说“失败了”，高质量错误消息会说明哪个字段不存在、哪个过滤值不合法、权限哪里受限、查询为何无法解析，以及下一步可尝试的修复方向。对 agent 来说，错误消息就是下一轮行动的观测输入。观测越具体，下一步越可能有效。

易错点

常见误区：很多团队把错误恢复看成 prompt 技巧，实际还要改工具接口和系统返回值。若工具只返回模糊异常，LLM 再强也只能猜；若工具返回结构化、可行动的失败原因，agent 才能把失败转成下一步计划。

6.4 Evals 中的质量跃迁

演讲者说，仅仅是错误恢复预算和高质量错误消息，就让质量分数 dramatic increase；许多更困难的 evals 明显变好。这里的 eval 同时承担最终答案评分和工程回归测试：同一类复杂问题，在 harness 改动前后是否更稳定、更能恢复、更少卡在可解释错误上。

这给分析 agent 一个朴素的工程原则：不要只评估“答对了吗”，也要评估“失败时如何失败”。在真实企业数据环境里，权限、枚举值、表关系、字段废弃、查询复杂度都会制造失败。一个可靠 agent 的价值，常常体现在它把失败变成可诊断、可修复、可继续推进的中间状态。

背景知识

质量跃迁的来源：模型没有凭空变聪明。系统把失败信息变得更可读，把修复空间显式留给 agent，于是同一个模型能完成更多复杂任务。这是 harness 工程对模型能力的放大。

6.5 从 Haiku 到 Sonnet：复杂对话的模型选择

早期处于问答模式时，Omni 使用 Haiku。Haiku 适合轻量、短上下文、低成本的交互。但进入更 elaborate 的 agentic conversations 后，对话更长，状态更多，工具结果更多，错误恢复路径也更多。演讲者说 Haiku 面向这种复杂会话时承载力不足，于是他们切换到 Sonnet。

切换后，图中展示了 token consumption 的变化。token 增长有两层含义：一方面，agentic 对话更长、更复杂，消耗更多 token 是设计结果；另一方面，Sonnet 带来了显著 unlock。客户开始反馈：他们问出过去自己无法回答的问题，或者即使能答也要花数小时的问题，Blobby 两分钟左右就命中了答案。此后 Blobby 的使用量开始快速增长。

Switched from Haiku → Sonnet

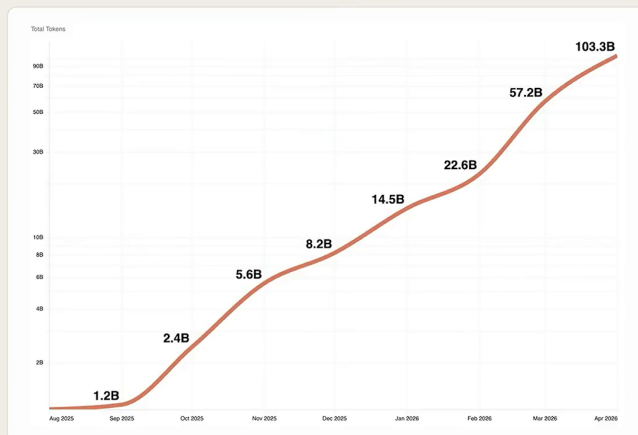


图 9: 从 Haiku 切换到 Sonnet 后, token 消耗随更长的 agentic conversation 和更高客户使用量一起上升。⁹

关键概念

模型选择原则:短问答可以优先考虑低延迟、低成本模型;多步工具循环需要更强的上下文承载、规划和修正能力。模型成本要和任务复杂度一起看,单看单次调用价格会低估 agentic workflow 的收益。

6.6 本章小结

第二阶段把 Blobby 从“能回答”推向“能执行任务”。agentic harness 引入任务循环,错误恢复预算给系统留下修复空间,高质量错误消息把失败变成可行动信息,evals 则记录这些改变是否真的提升复杂问题表现。Haiku 到 Sonnet 的切换说明,模型选择要匹配交互形态;当系统进入长对话、多工具、多轮修正时,更强模型可能直接解锁新的用户价值。

7 第三阶段: Trace 诊断与“合并大脑”

7.1 CEO 压力如何推动质量工程

使用量上升之后,Omni 的 CEO Colin 成了最响亮、最挑剔的用户。他的反馈很直接:这个东西很好,但某个问题答错了,去修。团队起初用 LLM 的不确定性来解释:模型有时难以完全稳定。CEO 的回应是“还不够好,去修”。这段压力很有工程含金量,因为它把“AI 偶尔会错”的宽泛说法,压缩成“请定位具体错误并提升系统”的质量任务。

这也是产品化 AI 与演示型 AI 的分界。演示型系统可以用随机性解释失败;客户环境中的分析系统要面对高价值问题,错误会影响判断、会议和决策。Omni 后续投入 trace 和 blobotomies,正是

⁹视频画面时间区间: 00:12:20-00:13:01。若为裁剪图,时间区间仍对应原视频画面。

这种压力推动出来的质量工程。

原始片段

00:13:15–00:13:40

CEO 的反馈可以概括为：它已经很有用，但这个问题答错了，团队需要修好。团队用 LLM 不稳定解释时，得到的回答是：这个理由不够。

7.2 Trace: 观察 Agent 内部对话

Omni 接下来重点投入 **trace**。trace 是一次 agent 会话的运行轨迹，包含模型消息、工具调用、工具返回、错误、观察、重试和中间判断。它像飞行记录仪：飞机落地成功时，它能解释系统怎么飞；飞行异常时，它能告诉工程师在哪个高度、哪个操作、哪个传感器读数出了问题。

对于 agentic loop，trace 可以形式化为一个事件序列：

$$\tau = (e_1, e_2, \dots, e_n), \quad e_i = (a_i, o_i, t_i)$$

其中：

- τ ：一次 agent 会话的 trace。
- e_i ：第 i 个运行事件。
- a_i ：agent 当时采取的动作，例如生成计划、调用工具、重试查询。
- o_i ：系统观察到的结果，例如工具返回、错误消息、查询结果。
- t_i ：事件发生的时间或顺序位置。
- n ：这一轮会话记录的事件数量。

有了 trace，团队可以看到 agent 如何“跟自己说话”、如何在循环中反应和回应。那些表面看似随机的坏会话，往往能在 trace 中暴露结构性根因：某个工具边界不合理、某个子任务被错误拆分、某类错误消息不足、某个 agent 拿不到关键上下文。

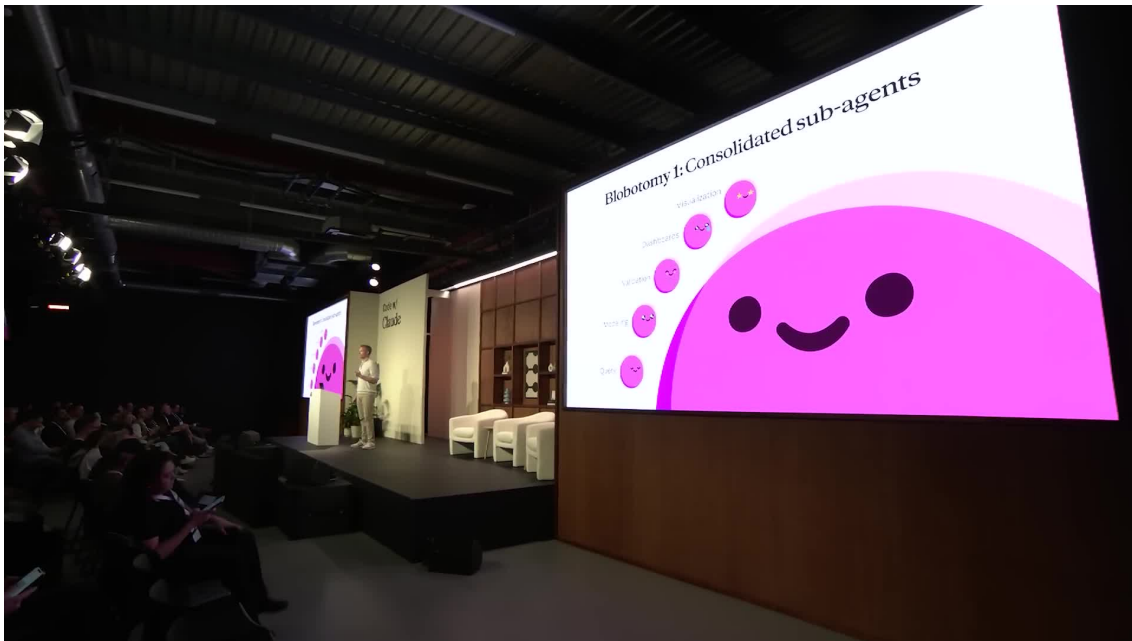


图 10: 第一次 blobotomy 将问题聚焦到 subagent 合并，暴露出外层规划与内层执行之间的信息边界风险。¹⁰

7.3 Blobotomies: 从失败会话定位结构病灶

演讲者把一系列重大修复称为 **blobotomies**。这个词带一点玩笑意味，借用了 lobotomy 的外科手术隐喻：修复对象已经深入到 Blobby 的“脑结构”。这里要抓住重点：blobotomy 属于 trace 驱动的架构修复，来源是运行轨迹里看到的真实失败模式。

当团队阅读 trace 时，原本被归因于“LLM 随机性”的失败开始变得具体。有些失败来自任务被拆给了拿不到全局信息的 subagent；有些错误来自外层 agent 不理解单条查询的可回答边界。trace 把模糊抱怨变成可定位的结构病灶。

关键概念

Blobotomies 的定义：blobotomies 是 Omni 基于 trace 对 Blobby 架构做的一系列重大修复。它们的目标是处理会话失败背后的结构性原因，避免把修复范围局限在单个失败样例的 prompt 上。

7.4 Outer Agent 与 Subagent 的 Split Brain

演讲者给出的核心例子，是早期 agent 设计有些“过于聪明”。系统里有一个 **outer agent**，负责生成 task list，并且理解可用数据在哪里；另有一个 **subagent**，负责 query generation。这个设计看起来合理，也便于把查询生成子 agent 复用到其他上下文。

问题出在信息边界。subagent 的职责是根据请求生成一条查询；outer agent 却不知道什么问题能在一条查询里回答。于是 outer agent 可能要求 subagent 同时回答 GitHub pull requests、support data，并把这些东西汇总。subagent 只能回应：这无法用一条查询完成，需要多条查询。

¹⁰ 视频画面时间区间：00:13:58-00:14:22。若为裁剪图，时间区间仍对应原视频画面。

这就是 **split brain**，即“大脑分裂”：外层 agent 和子 agent 分别掌握不同的知识边界，导致规划者不知道执行者的真实能力，执行者又缺少规划者的全局目标。类比到公司协作，就是项目经理知道客户目标，数据库工程师知道查询约束，两个人之间只传一句含混任务，返工几乎必然发生。

易错点

split brain 的失败模式：当 A_o 表示外层 agent， A_i 表示内层 subagent，如果 A_o 掌握任务目标但不了解查询可执行边界， A_i 掌握查询能力但缺少全局拆解权，系统就容易生成“看似合理、实际不可执行”的任务。

7.5 Consolidating the Brain: 把工具上提到外层 Harness

Omni 工程师 Joel 把最终修复称为 **consolidating the brain**，也就是“合并大脑”。做法是把关键工具能力上提到 outer agent harness，让外层 agent 直接拥有更完整的执行视野。这样它在制定任务时，就能同时理解数据可用性、查询生成能力和单条查询边界。

这个设计调整减少了很多看似不可预测、令人惊讶的行为。这条经验没有把所有 subagent 都判定为问题来源；真正的原则是，系统切分 agent 边界时，必须同步切分知识、权限和责任。某个组件如果要规划任务，就必须知道足够多的执行约束；某个组件如果只生成局部结果，就不能被要求承担跨数据域的综合推理。

关键概念

Consolidating the Brain 的定义：合并大脑是把关键工具和执行知识上提到外层 harness，减少 outer agent 与 subagent 之间的信息断裂。它的目标是让负责规划的层也理解可执行边界，从而降低复杂任务中的异常行为。

7.6 本章小结

第三阶段说明，AI agent 的质量提升需要可观测性。CEO 的苛刻反馈迫使团队从“模型不稳定”的泛泛解释，走向 trace 驱动的结构诊断。trace 让团队看到 agent 内部循环，blobotomies 则把失败会话转化为架构修复。outer agent 与 subagent 的 split brain 暴露了 agent 边界设计的风险；合并大脑把关键工具上提到外层 harness，使复杂 evals 获得明显改善。真正可维护的 agent 系统，需要把失败记录下来、看清楚，再改到结构上。

8 第四阶段：让 Claude 写 SQL，再由 Omni 解析

8.1 为什么重新启用 SQL Parser

这一阶段的动机很直接：团队发现 Claude 直接生成 SQL 时，能回答一些 Blobby 当时处理得很吃力的问题。这里的关键并非“SQL 更时髦”，关键在于 SQL 本身已经是一种成熟的分析表达语言。它能描述过滤、聚合、连接、分组、排序、窗口计算和分步推导，像一张数据问题的施工图。自然语言问题进入系统后，如果过早被压成一个狭窄的专有结构，很多原本可以被 SQL 表达的分析路径会被截断。

SQL Parser 是什么

SQL parser (SQL 解析器) 负责把一段 SQL 文本拆成机器可理解的结构, 例如抽象语法树、字段引用、表引用、过滤条件、聚合逻辑和依赖关系。直观地说, parser 像海关检查站: SQL 是旅客携带的行李, parser 要打开行李箱, 确认里面有哪些对象、关系和操作, 再决定能否把它安全映射进 Omni 的语义层和查询执行系统。

有意思的是, Omni 很早就构建过一套完整的 SQL parsing engine, 后来又把它搁置了。原因很现实: 人类用户会把各种随机 SQL 丢给系统, 方言差异、嵌套写法、边缘语法和不可预期的排列组合让 parser 难以稳定承接。一个企业级 parser 面对开放世界 SQL, 就像要解析所有人口音、俚语和临场发挥的翻译器, 工程成本会迅速膨胀。

Claude 改变了这个前提。模型生成的 SQL 虽然自由, 但它可以被提示词和示例约束到一个大致稳定的形状。于是旧 parser 的价值重新浮现: 它不再需要接住全世界任意人类 SQL 的全部变化, 只需要接住 Claude 在 guidelines 下生成的、Omni 能理解的 SQL 子集。演讲者还点名说, 工程师 Stephen 把这套搁置多年的解析代码重新拿出来, 并从根本上改变了 Blobby 暴露 query generation 能力的接口。

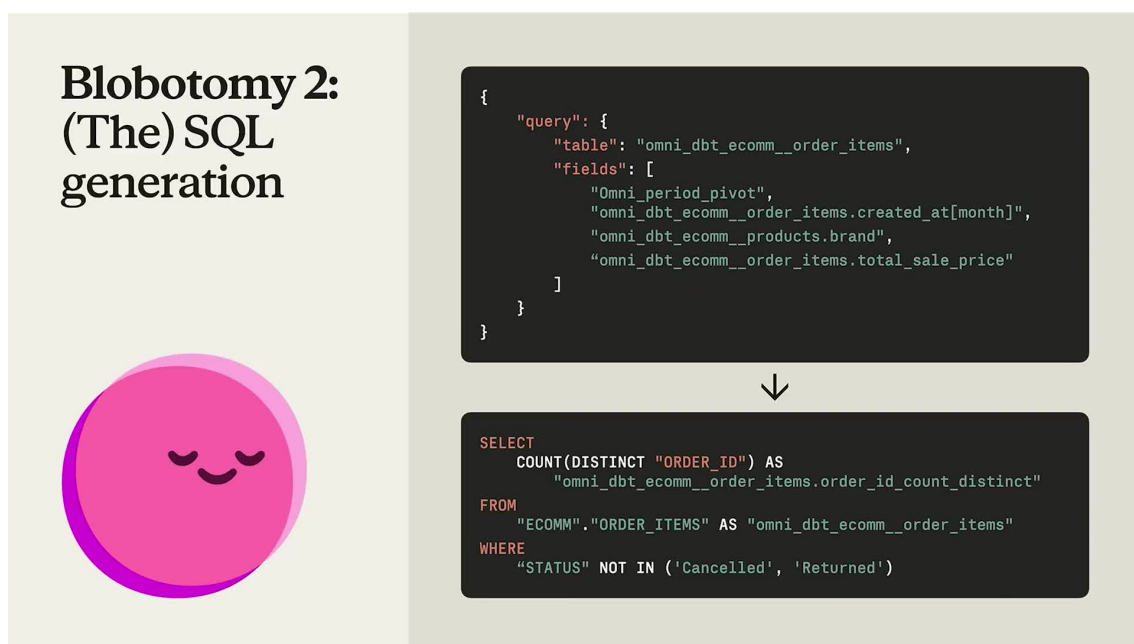


图 11: SQL generation blobotomy 把专有 JSON 查询表示换成 Claude 更擅长书写、Omni 又能解析的 SQL。¹¹

8.2 从 JSON 化查询切换到 SQL Parsing Mode

在这个转向之前, Blobby 暴露出的查询生成界面更像一种 JSON 化查询。JSONified query 指把查询意图放进一套专有 JSON 结构里, 例如字段、过滤、分组、排序分别进入固定键位。它的好处是边界清晰、类型明确、便于程序消费; 代价是模型必须先学习 Omni 发明的中间格式。对 LLM 来说, 这相当于让一个已经熟悉自然语言和 SQL 的分析师, 先背一套公司内部表格填报规范, 再开始写分析。

¹¹ 视频画面时间区间: 00:16:48-00:19:03。若为裁剪图, 时间区间仍对应原视频画面。

JSON 化查询与 SQL Parsing Mode 的差别

JSON 化查询强调结构先行：系统先规定槽位，模型把意图填进去。SQL parsing mode 强调语言先行：模型直接写 SQL，系统再解析 SQL，抽取出 Omni 需要的结构。前者像填报销单，后者像写一份分析草稿后交给审阅器拆解。选择 SQL parsing mode 的核心理由，是 Claude 已经具备强 SQL 先验，系统可以顺势借用这个能力。

SQL parsing mode 的工作方式可以理解为三步。第一，Claude 根据用户问题、语义层上下文和可用字段写出 SQL。第二，Omni 的 parser 读取 SQL，把它还原成系统内部可执行、可校验的结构。第三，harness 把解析结果交给查询执行、可视化或后续验证工具。这样一来，Claude 的表达空间扩大了，Omni 的执行边界仍然保留。

这里还有一个容易被忽略的工程收益：减少模型学习负担。JSON 化查询是 Omni 自己的协议，Claude 需要在 prompt 里被反复教会。SQL 是大模型训练中高频出现的公共语言，Claude 已经知道大量惯例。把接口切回 SQL，等于把系统接口移动到模型最熟练的地带。

8.3 Parser Guidelines: 约束自由 SQL 的边界

让 Claude 写 SQL 并不等于把仓库完全交给自由文本。Parser guidelines 的作用，就是给 Claude 的 SQL 生成划出可解析、可执行、可恢复的边界。它们通常会限制 SQL 形态，例如偏好某些 join 写法、避免 parser 不支持的方言特性、约束嵌套层级、规定字段引用方式，或者要求用明确别名表达中间结果。

自由表达需要工程边界

LLM 擅长生成看起来合理的 SQL，但生产系统关心的是可解析性、权限边界、稳定执行和错误恢复。parser guidelines 就像铁路轨道：列车仍然可以高速运行，但方向、轨距和岔路必须受控。没有 guidelines，SQL parsing mode 很容易退化成又一个开放世界解析问题。

这种设计的精妙之处在于，它没有试图压低 Claude 的 SQL 能力，而在于把能力导入一条 Omni 能消费的通道。约束并不必然降低智能；在工程系统里，约束经常是让智能产生可复现价值的前提。若把 Claude 看成强表达器，把 parser 看成结构化接收器，那么 guidelines 就是两者之间的协议层。

8.4 One-shot Query 与 CTE 带来的效率

切到 SQL parsing mode 后，一个明显变化是 Blobby 可以把过去需要三四次尝试、或者需要串联多条查询才能完成的问题，改写成 one-shot query。One-shot query 指一次生成并执行的查询，尽量在单条 SQL 中完成问题拆解、数据拉取、聚合和结果组织。它减少了 agent loop 中的往返次数，也降低了中间状态错配的概率。

演讲者特别提到 Claude 喜欢写带 CTE 的 SQL。CTE，全称 Common Table Expression，常见语法是 `WITH name AS (...)`。它把复杂查询拆成多个有名字的中间结果，像在黑板上先写“第一步得到候选集合”、“第二步按客户聚合”、“第三步计算排名”，最后再组合答案。对人类读者来说，CTE 让 SQL 更像分步骤推理；对 parser 来说，规则清晰的 CTE 也更容易拆解。

CTE 的直觉

CTE 可以被看作 SQL 里的临时便签。每张便签记录一个中间表，后面的 SQL 可以引用它。相比把所有逻辑塞进一层巨大的嵌套查询，CTE 更接近分析师写草稿的方式：先定义局部结果，再把局部结果拼成最终答案。

从效率角度看，收益至少有三层。第一，模型不必生成专有 JSON 结构，prompt 和推理成本下降。第二，单条 SQL 可以表达更复杂的依赖关系，减少多轮查询和多轮修补。第三，CTE 把复杂逻辑分块，既方便 Claude 组织答案，也方便 parser 和后续 validation 工具定位错误。

8.5 押注 Claude 的 SQL 能力

Omni 的这次改造还包含一个产品判断：Anthropic 会持续投入，让 Claude 更擅长 SQL。这个判断非常务实。企业分析场景里，SQL 是通往数据仓库的主干道；任何通用模型若想服务数据工作流，都绕不开 SQL 生成、SQL 修改、SQL 解释和 SQL 调试。把系统接口押到 SQL 上，等于把 Omni 的工程演进和 Claude 的模型能力演进接上了同一条增长曲线。

这类押注有风险。模型能力提升并不会自动带来系统质量提升；如果没有 parser、guidelines、权限控制、trace 和 eval，SQL 生成再强也可能制造不可预测结果。Omni 真正做对的地方，是把 Claude 的强项放进一个可检查的 harness：生成交给模型，解析交给 parser，边界交给 guidelines，质量交给 eval 和 trace 追踪。

8.6 本章小结

第四阶段的核心变化，是把 Blobby 的查询生成接口从专有 JSON 化结构切换到 SQL parsing mode。旧 SQL parser 曾因开放世界 SQL 过于复杂而被搁置；Claude 能在 guidelines 下生成较规整 SQL 后，parser 重新成为可用资产。CTE 与 one-shot query 进一步提升了效率：复杂问题可以在一条更自然、更可解析的 SQL 中完成，减少 agent 往返和中间状态漂移。

9 当前 Harness：Checkpoint、工具面与 Evals

9.1 Outer Loop：Checkpoint 与故障恢复

演讲进入当前架构时，重点从“某一次查询如何生成”转向“整个 agentic system 如何可靠运行”。Omni 现在的系统包含一个 outer loop，它负责 checkpointing。Checkpoint（检查点）指在执行过程中保存关键状态：当前任务、已调用工具、已得到结果、错误信息、下一步计划等。若中途失败，系统可以从最近的检查点恢复，而不必从用户问题的起点重新跑完整流程。

Checkpoint 的工程含义

Checkpoint 把一次 agent 执行从“一次性对话”变成“可恢复流程”。这对分析系统尤其重要，因为一次 dashboard、validation 或 data modeling 任务可能包含多次工具调用。没有 checkpoint，任意一步失败都可能浪费前面已经完成的探索、查询和推理。

可以用一个紧凑公式来描述这一代 harness：

$$H = (M, T, K, \tau, E, B_e)$$

其中各符号含义如下：

- H : harness, 即围绕 Claude 构建的 agentic 执行框架。
- M : model, 被调用的 Claude 模型。
- $T = \{t_1, \dots, t_n\}$: tool set, 可调用工具集合, 例如查询、dashboard、visualization、validation、data modeling。
- K : checkpoint state, 保存任务状态、已调用工具、已有结果、错误信息和下一步计划。
- τ : 执行轨迹, 记录模型思考、工具调用、错误、恢复和结果。
- E : eval system, 用来度量质量并提供可观测性。
- B_e : 错误恢复预算, 表示允许 agent 为修复失败消耗的轮次、时间或 token 空间。

这个公式的重点, 是提醒读者不要把 agentic analytics 理解成“模型加提示词”。真正的产品 harness 是模型、工具、检查点、轨迹、评估和恢复预算的组合系统。Parser guidelines、权限和语义定义仍然构成重要执行边界, 但它们分别约束 SQL parser、semantic layer 和工具调用, 不能和 B_e 混成同一个符号。

9.2 Inner Loop: 不断扩展的工具集合

outer loop 负责恢复和协调, inner loop 则负责具体行动。演讲者说, 当前系统的 inner loop 中已经有许多工具, 而且工具面还在快速扩大。Tool set 指 agent 可以调用的一组外部能力: 查语义模型、生成查询、执行查询、创建 dashboard、生成 visualization、做 validation、修改 data model 等。

工具集合的扩张意味着 Blobby 的角色正在变化。早期它更像回答问题的接口; 现在它更像有工具箱的数据分析同事。用户提问后, 它可以选择合适工具, 观察工具返回, 再决定下一步。这里的复杂性在于工具越多, 选择空间越大, 错误路径也越多。因此 checkpoint、trace 和 eval 的重要性会随工具面一起上升。

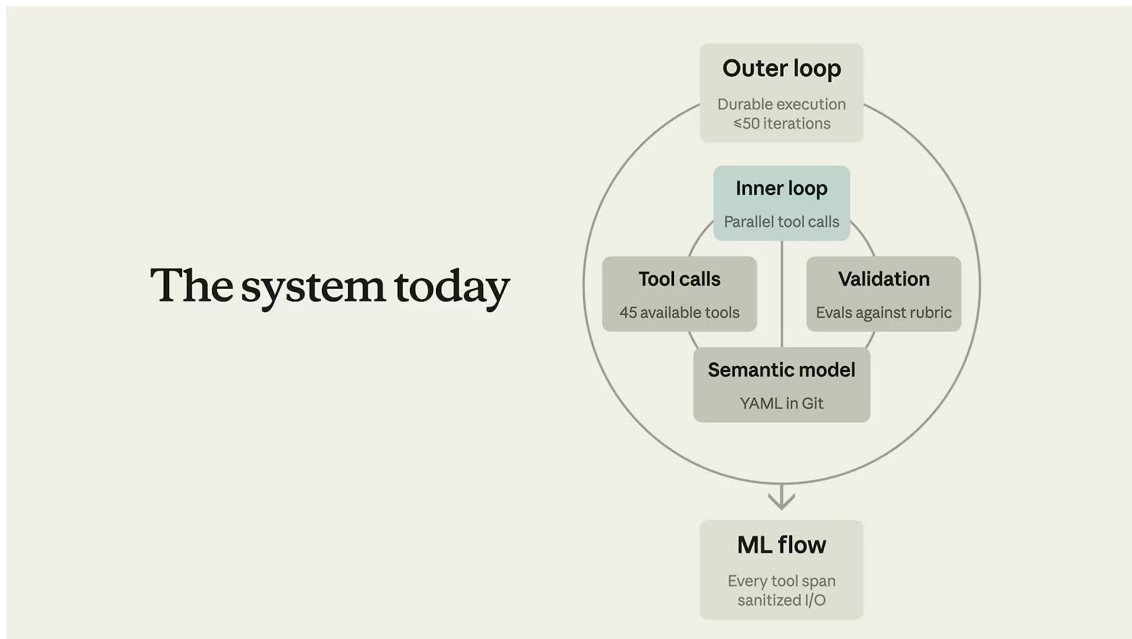


图 12: 当前 harness 用外层持久执行循环承载 checkpoint，用内层工具循环处理查询、验证、语义模型访问和并行工具调用。¹²

工具越多，harness 越重要

增加工具并不会自动提升 agent 质量。工具集合扩大后，系统更需要清楚的工具描述、调用前置条件、错误消息、恢复策略和 trace 记录。否则 agent 会在多个可行路径之间摇摆，表现出用户眼里的“不稳定”。

9.3 Dashboard、Visualization、Validation 与 Data Modeling

演讲者列举了几个重要工具方向。Dashboard 工具让 Blobby 不只返回一张结果表，还能组织多个查询和图表，形成可复用的分析页面。Visualization 工具把查询结果转成图形表达，让趋势、分布、异常和对比更容易被人理解。Validation 工具负责检查答案是否可靠，例如字段是否用对、过滤条件是否符合问题、结果是否自治。Data modeling 工具则更进一步，让 Blobby 能改善语义层本身。

这些工具覆盖了分析工作的完整链条：从找数据，到算结果，到解释结果，到检查结果，再到改进数据模型。它们对应的质量指标也不同。查询工具看重正确性和可执行性；可视化工具看重表达是否贴合问题；validation 工具看重发现错误的敏感度；data modeling 工具看重长期语义资产是否变好。

Predictability and Quality

Predictability（可预测性）强调同一个关键问题在相同上下文下应得到稳定答案；quality（质量）强调答案本身正确、完整、可解释、符合业务定义。CEO 问一个关键指标时，系统既要答对，也要在重复询问时保持一致。前者像罗盘不乱跳，后者像罗盘确实指向北。

¹² 视频画面时间区间：00:19:04-00:20:11。若为裁剪图，时间区间仍对应原视频画面。

9.4 Evals 的双重价值：质量度量与可观测性

Omni 同时建设内部 eval system 和面向客户的 eval system。Eval 是一组用来测试 agent 行为的评估任务，通常包含输入问题、期望行为、评分标准和执行 trace。Judge 则是把评估标准自动化的评分器，可以由程序规则、模型评分或二者组合完成。它把“这次回答好不好”从主观感觉变成更可重复的判断流程。

演讲者对 eval 的偏爱有一个很具体的理由：他最看重 raw trace data。很多团队谈 eval 时首先想到分数，例如通过率、准确率、回归率。Omni 的经验补充了另一面：eval 还是可观测性工具。失败样本背后的 trace 能揭示 agent 到底在哪一步偏离：选错字段、误解业务词、生成了 parser 不支持的 SQL、工具错误没有恢复，或者 judge 标准本身需要调整。

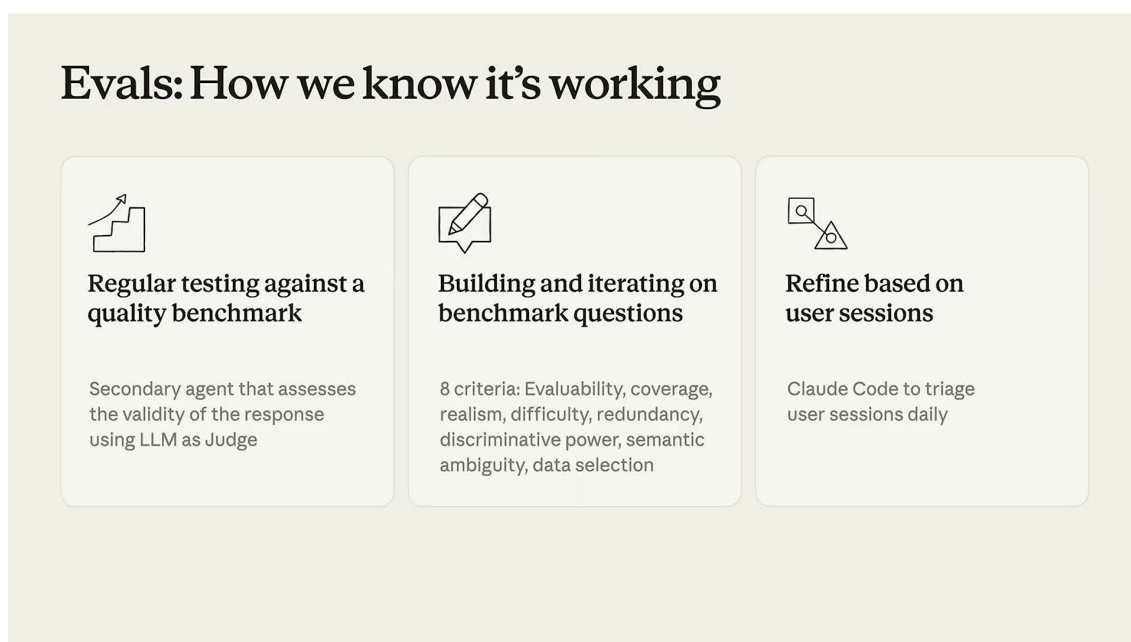


图 13: eval 流程把质量基准、问题设计和每日用户会话诊断串起来，使评分与可观测性同时服务质量改进。¹³

Evals 的双重价值

Eval 的第一层价值是度量质量，帮助团队知道改动是否提升了正确率和稳定性。第二层价值是暴露过程，让工程师看到失败如何发生。分数告诉团队哪里疼，trace 告诉团队病灶在哪。

把失败 trace 捕捉进 judge，则带来效率收益。人工先读 trace、总结失败模式，再把稳定模式沉淀成 judge。下一次类似问题出现时，系统可以自动识别。这是一种从观察到规则的压缩过程：先用人类判断理解问题，再用自动化判断扩大覆盖面。

9.5 Claude Code 如何反向塑造产品 Harness

最后一个观点很关键：Omni 工程师使用 Claude Code 的经验，反过来帮助他们理解该如何构建 Blobby 的 harness。一个系统的设计者若无法想象用户为何需要某个能力，就很难做出顺手的产品。工

¹³ 视频画面时间区间：00:20:11–00:20:55。若为裁剪图，时间区间仍对应原视频画面。

工程师每天使用 Claude Code，会自然感受到一个好 harness 应该怎样探索代码库、怎样调用工具、怎样展示中间过程、怎样恢复错误、怎样把上下文放在合适位置。

演讲者举了一个类比：semantic model 和 codebase 有相似之处。代码库里有文件、函数、依赖和局部上下文；语义模型里有字段、表、关系、权限和业务定义。Claude Code 探索代码库的方式，可以启发 Blobby 探索语义模型的方式。换句话说，Omni 不只是用 Claude Code 提升开发效率，也把 Claude Code 暴露出的产品模式转译进自己的分析 agent。

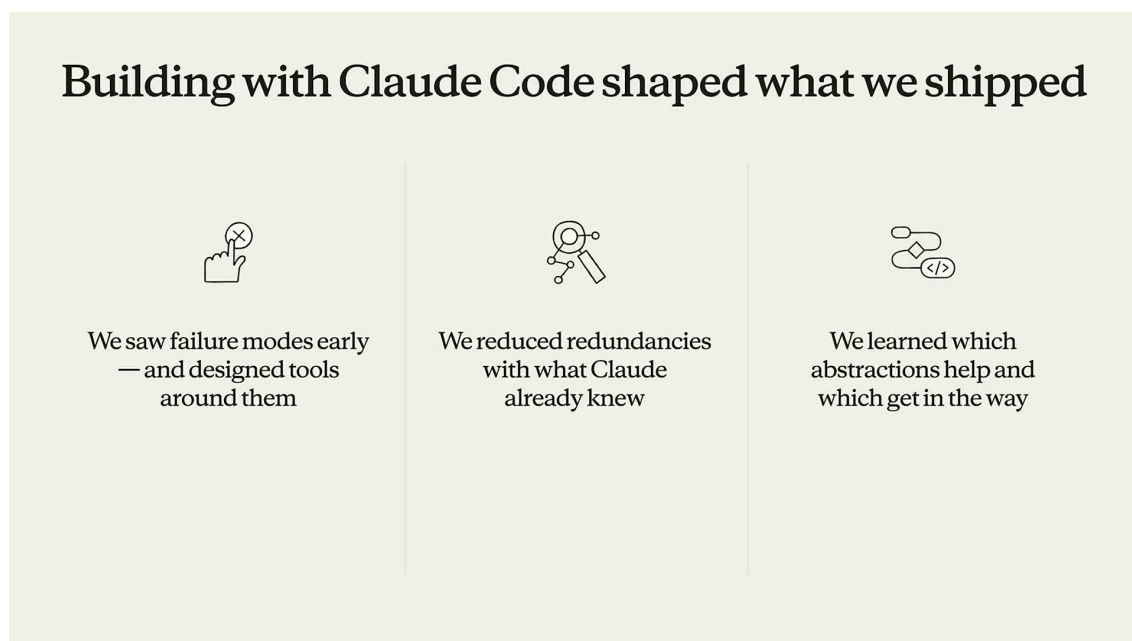


图 14: 使用 Claude Code 的经验反向塑造了 Omni 自身 harness 的设计，使工程团队更早看到边界、抽象和工具反馈的实际效果。¹⁴

产品 harness 的反向学习

Claude Code 对 Omni 的影响有两层：一层是工程生产力，另一层是产品直觉。前者让团队写得更快，后者让团队更懂 agentic harness 应该怎样被用户感知、检查和控制。后者更隐蔽，也更有长期价值。

9.6 本章小结

当前的 Omni harness 可以被理解为一个带 checkpoint 的外层循环，加上不断扩张的内层工具集合，再加上 trace 和 eval 构成的质量闭环。Dashboard、visualization、validation 和 data modeling 让 Blobby 从问答工具走向分析执行系统；predictability and quality 则定义了客户真正关心的底线：关键问题必须答对，并且重复询问时保持稳定。Claude Code 的使用经验进一步塑造了 Omni 对 harness 的产品判断，让工程师把自己作为 agent 用户得到的直觉，转化为 Blobby 面向数据分析场景的能力。

¹⁴ 视频画面时间区间：00:20:55–00:21:59。若为裁剪图，时间区间仍对应原视频画面。

10 现场演示：AI 生成、UI 校验与排障

10.1 创建工程活动 Dashboard

演讲者把前面二十多分钟的架构讨论切到现场演示模式，目标很具体：让 Blobby 创建一个关于 Omni 代码仓库工程活动的 dashboard。现场输入近似于 `create a dashboard of engineering activity in the omnipository`，随后系统需要创建多条查询，并思考 dashboard 如何布局，所以运行会花一点时间。这个动作看似只是一条自然语言指令，背后实际触发了一组查询生成、布局规划、数据主题选择和结果组织步骤。换句话说，用户输入的是一句业务问题，系统要交付的是一个可浏览、可追问、可修改的分析页面。

关键概念

这里的 dashboard 指产品 workflow 中的“分析成品页”：它把多个指标、图表和摘要组织在同一个页面里，帮助用户从不同角度观察同一类业务现象。它远超单张图的范围，更像一张可以继续钻取的分析看板，里面包含若干查询、过滤条件、图表布局和文字解释。

这段演示也把全片的技术主线落回用户体验。前面的章节已经解释过语义层、SQL parser、agentic loop、checkpoint、trace 和 eval；现场演示关心的是这些机制合在一起之后，用户是否能把一句“看一下工程活动”变成可以检查的数据资产。

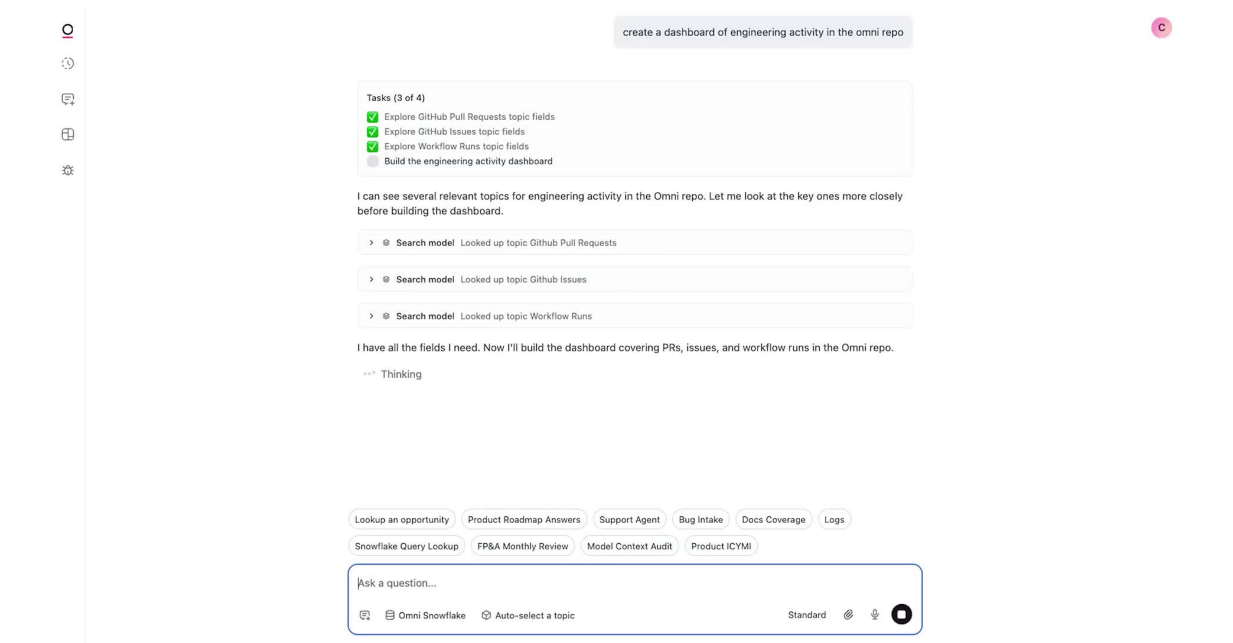


图 15: 现场演示中，agent 先识别与工程活动 dashboard 相关的 GitHub topics，并把任务进度呈现在界面中。¹⁵

10.2 Topic Discovery 与 Dashboard Plan

Blobby 首先寻找相关 topic，并生成 dashboard plan。演讲者解释说，Omni 中的 topic 可以理解为一个数据领域，近似于一个宽表化、主题化的数据入口：它把许多子数据集组合到一起，让用户和

¹⁵ 视频画面时间区间：00:22:24-00:23:04。若为裁剪图，时间区间仍对应原视频画面。

agent 先站在“业务域”的层面思考，再进入字段、过滤器和具体查询。

背景知识

在产品工作流里，topic 的价值类似图书馆里的“主题馆藏”。用户不需要先知道每本书的架号，也不需要先背出底层表名；系统先找到与问题相关的数据区域，再把查询落到具体字段与关系上。对于 agent 来说，topic discovery 就是把开放式自然语言问题收窄到可执行数据空间的第一步。

Dashboard plan 则是 agent 对最终页面的草图：需要哪些问题、每个问题需要哪些查询、哪些图表适合并排呈现、哪些摘要可以帮助读者理解结果。可以用一个简化公式描述这个过程：

$$\text{User Request} \rightarrow \text{Topic Discovery} \rightarrow \text{Dashboard Plan} \rightarrow \{q_1, q_2, \dots, q_n\} \rightarrow \text{Dashboard}$$

其中各符号含义如下：

- User Request：用户的自然语言目标，例如“创建工程活动 dashboard”。
- Topic Discovery：寻找相关数据主题，把问题绑定到可查询的数据领域。
- Dashboard Plan：规划 dashboard 的指标、图表、布局和分析顺序。
- $\{q_1, q_2, \dots, q_n\}$ ：支撑 dashboard 的一组查询。
- Dashboard：最终可预览、可检查、可继续编辑的分析页面。

10.3 AI 生成，UI 校验与排障

演讲者用一句话概括了 Omni 的产品哲学：AI to build, UI to validate and troubleshoot and refine。中文可译为“AI 生成，UI 校验、排障与精修”。这一句是本章的核心，因为它把 agent 产品从“聊天框回答问题”推进到“生成可审计分析对象”。

关键概念

“AI 生成，UI 校验与排障”指一种产品协作模式：AI 负责把模糊目标推进成查询、图表、dashboard 等可操作对象；UI 负责让人类检查过滤条件、查看数据形状、修改图表、定位错误并继续探索。AI 提供速度，UI 提供可见性、控制感和纠错入口。

这个原则有一个很现实的出发点：数据分析的错误经常藏在细节里。SQL 文本可能很长，过滤器可能很隐蔽，字段语义可能被误解，图表也可能因缺数据而空白。只给用户一个自然语言答案，会把这些可疑点压扁成一段看似流畅的文本。把结果打开到 UI 中，用户可以像检查电子表格一样逐层看见 agent 的假设。

易错点

Agentic analytics 的危险点在于“看起来已经完成”。自然语言摘要越顺滑，用户越容易忽略底层过滤条件、仓库范围、分支范围、合并状态和时间窗口。Omni 的演示把 UI 放在流程中心，目的就是让用户能检查这些细节。

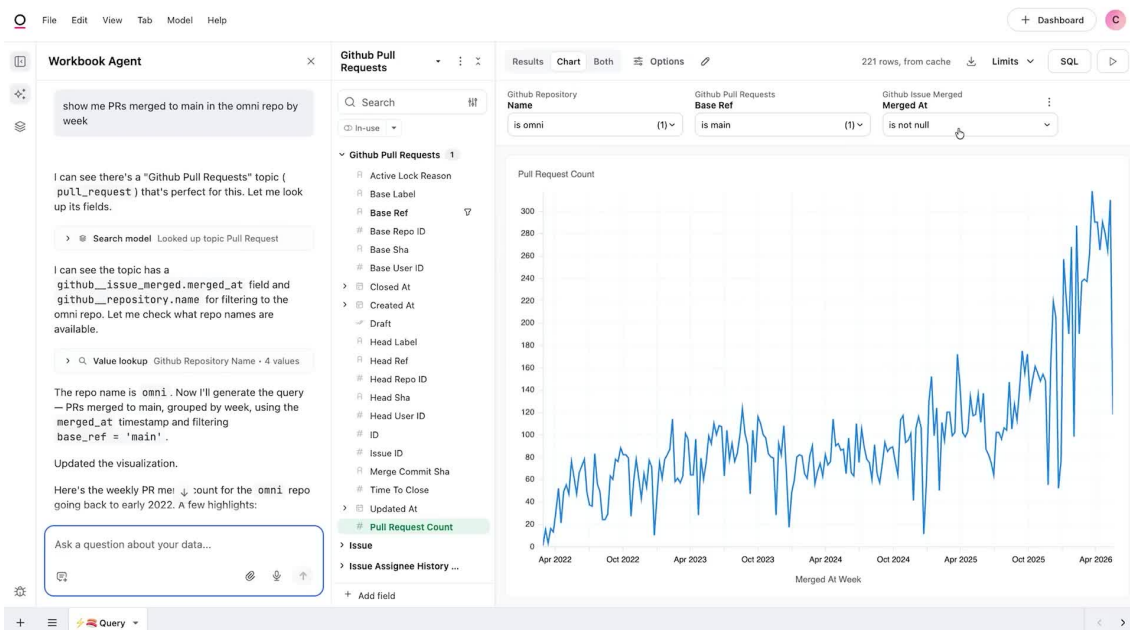


图 16: Workbook 视图让 agent 的结果可检查: repository、main 分支、已合并 PR 等过滤条件和趋势图同时可见。¹⁶

10.4 Workbook 中检查 PR 数据与过滤条件

演讲者接着打开 workbook，检查 Blobby 生成的查询结果。Workbook 在 Omni 里是一个分析工作簿，用户可以在里面生成查询、查看数据、操作图表并修改条件。它承担的是“从 agent 产物进入人工检查”的桥梁角色。

背景知识

Workbook 可以类比为分析师的工作台。Dashboard 偏向展示结果，workbook 偏向检查和加工过程。用户在 workbook 中能看到数据集、过滤器、图表配置和查询结果，从而判断 agent 是否把问题理解对了。

现场检查的对象是 GitHub pull request 数据集。演讲者确认了几个关键过滤条件：数据集确实是 GitHub PR 数据，仓库过滤到了 Omni repository，分支过滤到了 main branch，PR 状态过滤到了 merged pull requests。这里每个条件都很小，却共同决定了图表能否回答“工程活动”这个问题。

把这个检查过程抽象成一个数据验证函数，可以写成：

$$V(R) = \mathbf{1}[d = \text{GitHub PR} \wedge r = \text{Omni repository} \wedge b = \text{main} \wedge s = \text{merged}]$$

其中各符号含义如下：

- R : agent 生成的查询结果或图表输入数据。
- $V(R)$: 对结果是否满足当前分析意图的校验函数。

¹⁶ 视频画面时间区间：00:23:36-00:24:39。若为裁剪图，时间区间仍对应原视频画面。

- d : 数据集类型，此处应为 GitHub pull request 数据。
- r : repository 过滤条件。
- b : branch 过滤条件。
- s : PR 状态过滤条件。
- $1[\cdot]$: 指示函数，条件成立时取 1，否则取 0。

公式只是把演示中的检查动作形式化，并不表示 Omni UI 里真的有这样一段代码。用户要验证的是一组被过滤条件约束的数据事实，而非孤立图表。

10.5 切换 Repository 与继续分析

演讲者随后演示了继续 “riff on what Blobby has done” 的过程。这里的 riff 可以理解为在 agent 已生成的基础上继续即兴分析：用户可选特定用户，也可以把 repository 切到 docs repository，快速观察同一套分析在另一个范围内的表现。

这一步展示了 workbook 的另一个价值：它让 AI 产物变成可变参数的分析对象。用户不用从零写 SQL，也不用完全接受 agent 的第一版结果；他可以沿着同一个分析结构调整过滤条件，比较不同仓库、不同作者、不同时间窗口的活动。

关键概念

好的 agent 产品应当把“生成结果”变成“生成起点”。Blobby 给出第一版查询和 dashboard，workbook 让用户继续改范围、换过滤器、看数据形状。这样，agent 的输出就从一次性回答变成可迭代的分析 workflow。

10.6 预览 Dashboard：指标、作者与 PR 趋势

回到 dashboard 后，Blobby 已经完成生成。演讲者看到摘要里出现 engineering activity 与 key metrics，于是打开预览。Dashboard 在分屏中呈现出来，包含过去三个月的 top PR authors、PR volume over time 等内容。演讲者还把时间窗口拉长到过去十二个月，以便呼应开头 slide 中关于工程活动趋势的说法。

这里有一个重要细节：dashboard 的价值不只在“自动画图”，还在于它把前面章节的系统能力串成了用户可见结果。语义层帮助系统找到 PR 与仓库含义，harness 组织多步查询和图表生成，工具集负责 dashboard 与 visualization，UI 则让用户修改时间窗口并观察趋势。

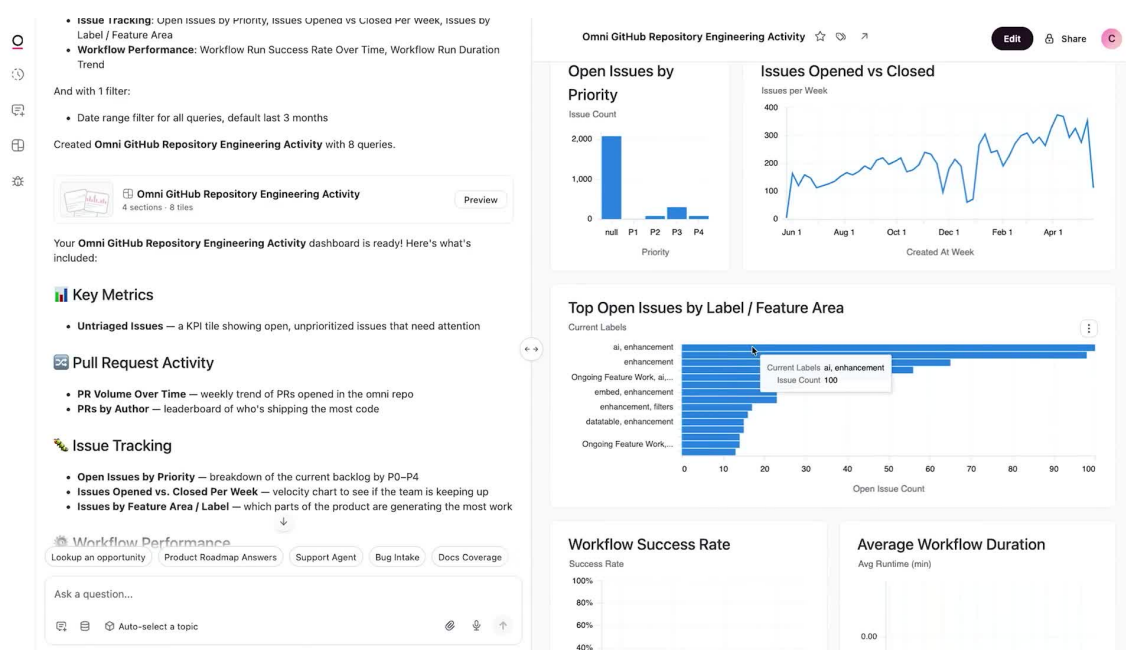


图 17: 生成的工程活动 dashboard 在 split pane 中展示指标、作者、PR 趋势和 workflow 区块，体现 agent 生成多图表分析产物的能力。¹⁷

演讲者还注意到“AI”是 Omni 内部很热的话题。这句话不应被过度解释成定量结论，因为字幕没有给出可读数值；它更像现场观察：dashboard 能把组织中的工程热点变成可见趋势，让团队围绕同一份数据讨论。

10.7 空白图表与现场排障

演示最后出现了一个很有教学价值的故障：workflow data 没有正确显示，某个图表是空白的。演讲者没有遮掩这个问题，并尝试现场排障，打开空白图表后承认自己一眼看不出原因，推测可能是这类数据没有很好地填充。

易错点

空白图表是 agentic analytics 中非常典型的失败形态。它可能来自数据缺失、过滤条件过窄、字段映射错误、图表配置错误、权限限制或时间窗口不匹配。现场演示中的空白图表说明，UI 校验属于把 agent 输出变成可信分析资产的必要环节。

这一刻也提醒读者：现场 demo 的瑕疵本身就是证据。若系统只能展示一段润色后的成功路径，用户很难判断它在真实工作中如何失败、如何恢复、如何让人类介入。Blobby 的空图表反而把 Omni 的产品原则讲得更清楚：生成只是第一步，检查、定位和修正同样属于分析系统的核心能力。

¹⁷ 视频画面时间区间：00:24:41–00:25:35。若为裁剪图，时间区间仍对应原视频画面。

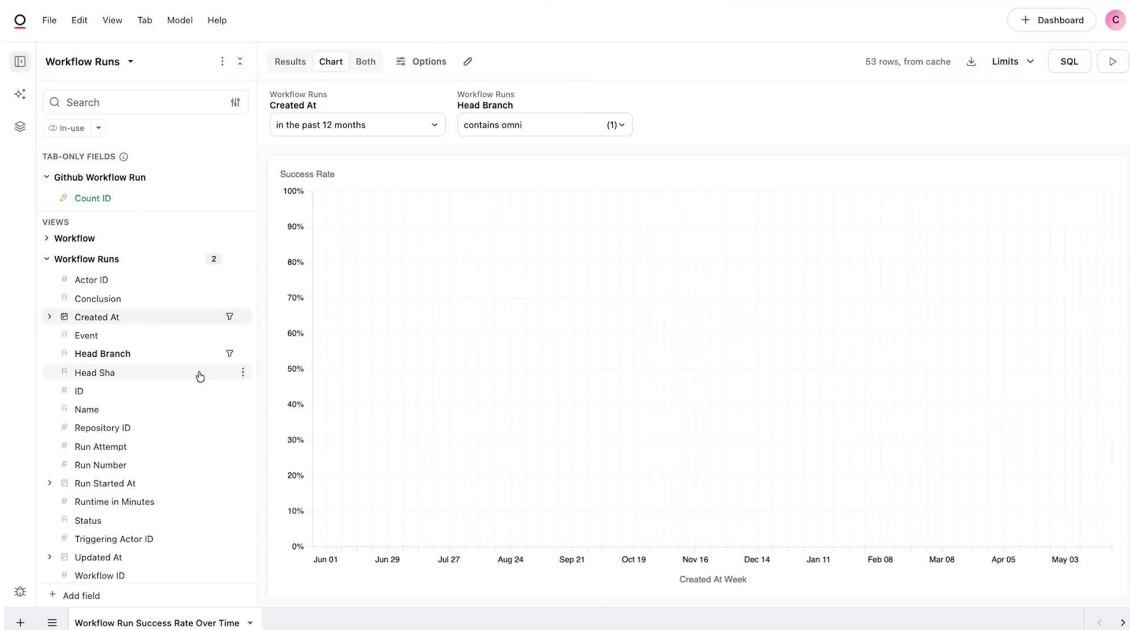


图 18: workflow success 图表为空白后，UI 成为排障表面，支持演讲者现场追问和检查数据是否真的存在。¹⁸

10.8 本章小结

本章把前面所有架构概念压到一个现场任务里：创建工程活动 dashboard。Bobby 先做 topic discovery 和 dashboard planning，再生成查询与图表；用户通过 workbook 检查 GitHub PR 数据集、repository、main branch 和 merged PR 过滤条件；随后用户切换 repository 继续分析，并在 dashboard 预览中观察作者、PR 趋势和时间窗口变化。

最关键的产品原则是“AI 生成，UI 校验与排障”。如果说语义层负责让数据“可被理解”，harness 负责让 agent “可执行且可恢复”，那么 workbook 与 dashboard UI 负责让分析“可被人类检查、修正和继续使用”。现场空白图表没有削弱这个原则，反而把它变得更可信：真实分析系统必须给失败留出可见入口。

11 总结与延伸：面向 Claude 优化的分析系统

11.1 演讲者的收束：Omni AI Analytics Platform Powered by Claude

演讲者最后把整场分享收束到一个定位：Omni 是一个 powered by Claude 的 AI analytics platform。更具体地说，Omni 团队有意把自己的 harness 设计成适配 Claude 以及 Claude family of models 的系统。他还提到 Omni 拥有优秀客户、工程团队和其他团队成员，团队位于 San Francisco，现场还准备了 Bobby 贴纸。

这些收尾信息里，真正和技术主线相关的是“optimized for Claude”。整场分享强调：把模型接进聊天框后等待奇迹发生远远不够，团队需要围绕 Claude 的能力边界设计语义层、工具、trace、eval、

¹⁸ 视频画面时间区间：00:25:36–00:26:08。若为裁剪图，时间区间仍对应原视频画面。

SQL 接口、checkpoint 与 UI 工作流。模型是核心推理引擎，产品系统则负责把推理转化为稳定、可调试、可复用的分析能力。



图 19: 收束页把 Omni 定位为 Claude 驱动的 AI analytics platform，并展示客户、团队规模和公司背景。¹⁹

11.2 全链路压缩：语义层、Harness、工具与 Evals

全片可以压缩成一条分析链路：

$$Q \xrightarrow{M,H} q_s \xrightarrow{S} \text{SQL} \xrightarrow{D} R, \quad H = (M, T, K, \tau, E, B_e)$$

其中各符号含义如下：

- Q : 用户的自然语言问题。
- M : Claude 模型。
- H : agentic analytics harness。
- q_s : 语义查询或被语义层约束后的查询意图。
- S : semantic layer，包含业务定义、上下文、权限和字段值样本。
- SQL : 最终交给数据库执行的结构化查询。
- D : 数据仓库或底层数据库。
- R : 查询结果、图表、摘要或 dashboard。
- T : 工具集合，包括查询、visualization、dashboard、validation 和 data modeling 工具。

¹⁹视频画面时间区间：00:26:10–00:26:42。若为裁剪图，时间区间仍对应原视频画面。

- K : checkpoint, 用于保存状态并支持恢复。
- τ : trace, 用来观察 agent 的中间决策和工具调用。
- E : eval, 用来衡量质量、稳定性和回归。
- B_e : 错误恢复预算。

这条公式把各章连在一起：业务上下文通过 S 进入系统，执行可靠性由 H 提供，工具设计通过 trace τ 反复改进，产品可用性通过“AI 生成，UI 校验与排障”落到用户手中。它也解释了为什么 Omni 会从简单问答走向 agentic loop，从专有 JSON 查询切到 SQL parsing mode，从单次输出走向可观测 trace 和 eval。

11.3 从工程经验到产品原则

Omni 的经验可以概括为四条产品原则。

第一，模型需要业务语义。企业数据仓库像一座巨大的城市，表名、字段名和权限规则就是街道、门牌和门禁。Claude 可以推理，但它需要语义层告诉它哪些路可走、哪些字段可信、哪些过滤值合法。

第二，agent 需要 harness。Agentic loop 让系统能规划、执行、观察和修正；checkpoint、错误消息和错误恢复预算让失败具备继续推进的空间。没有 harness，复杂任务会退化成一次性模型调用。

第三，工具边界会改变智能形态。早期 subagent 与 outer agent 的 split brain 说明，工具放在哪一层、谁拥有上下文、谁能调用查询生成能力，都会影响 agent 是否能完成任务。把关键工具上提到外层 harness，本质上是在减少信息断裂。

第四，产品 UI 要承认 AI 会出错。Workbook、dashboard preview 和现场排障都在传达同一个判断：可信的 AI 分析系统应当让用户看见中间结果、修改参数、检查过滤器并处理空图表。

11.4 实践清单：团队可以复用的设计问题

下面这组问题适合任何正在构建 AI analytics 或数据 agent 的团队复用：

- 哪些业务上下文必须贴近数据定义保存，而不应只放在 prompt 里？
- 字段 label、description、AI context、sample queries 和 values 是否覆盖了真实用户会问的表达？
- 系统返回给 agent 的错误消息是否包含可修复线索？
- agent 失败时是否有 checkpoint 和足够的恢复预算？
- 哪些工具边界可能制造 split brain 风险？
- Claude 是否可以直接使用 SQL 这类更自然、更有表达力的语言，减少专有格式带来的摩擦？
- eval 是否同时暴露原始 trace、稳定 judge 和回归信号？
- UI 是否允许用户检查过滤条件、数据集、时间窗口、权限和空图表？
- Dashboard 生成后，用户能否把它作为继续分析的起点，避免只能阅读最终摘要？

11.5 开放问题与下一步

这份笔记需要保留一个限制说明和两个后续检查点。第一，本视频没有可用的 YouTube 人工字幕或自动字幕，当前文本依据 Faster-Whisper 英文转写生成。Whisper 字幕足以支撑章节级理解，但专有名词、产品名和现场口误仍可能存在误差，后续整合时应结合画面与人工复核修正。

第二，F18 到 F22 已经由 Figure Agent 2 视觉确认，且图片已接入正文；后续再版只需复核裁剪清晰度、caption 是否继续贴合 live demo 叙事，以及 closing slide 的客户和团队信息是否需要更新。

第三，现场空白图表应被当作设计证据保留下来。它揭示了真实数据产品里最常见的问题：agent 可以生成漂亮结构，但数据填充、过滤范围和图表配置仍需要检查。后续如果继续扩展这套系统，最值得投入的方向包括更好的空图诊断、更细的 trace UI、更稳定的 eval judge，以及让用户能把排障结果反馈回 semantic layer。

最终，Omni 这场分享给出的核心答案很直接：要让 Claude 成为分析系统的一部分，团队需要同时优化模型输入、工具接口、执行 harness、评测系统 and 人机协作界面。任何一个环节薄弱，都会把智能压回“看起来会说话”的演示；这些环节一起工作，AI 分析才有机会进入团队日常的工程和业务决策。